COURSE 1 - ANALYSIS AND DESIGN

1.1 Objective

The objective of this course is to make you familiar with the following terms:

- SDLC
- Requirement Analysis
- Software Design

1.2 Course Content

1.2.1 SDLC and different phases of SDLC

The software development life cycle (SDLC) is a framework which defines tasks performed at each step in the software development process.

1.2.1.1 What is SDLC?

SDLC is a process followed in a software project, within a software organization. It consists of a detailed plan describing how to develop, maintain, replace and alter or enhance **a** specific software. The life cycle defines a methodology for improving the quality of software and the overall development process.

1.2.1.2 Different SDLC Models - Waterfall, V Model, Agile

There are various software development life cycle models defined and designed which are followed during the software development process. These models are also referred as "Software Development Process Models".,
In order to ensure success in process of software development, each process model follows a series of steps unique to its type.

The following are the most important and popular SDLC models followed in the industry:

- **Waterfall Model** - The Waterfall model was first process model to be introduced. It is also referred to as a linear-sequential life cycle model. It is very simple to understand and use. In a waterfall model, the outcome of one phase acts as the input for the next phase sequentially.

- **Iterative Model** - In Iterative model, iterative process starts with a simple implementation of a small set of the software requirements **which** iteratively enhances the evolving versions until the complete system is implemented and ready to be deployed.
- **Spiral Model** - The spiral model combines the idea of iterative development with the

systematic, controlled aspects of the waterfall model.

- **V-Model** – It is is a SDLC model where execution of processes happen in a sequential manner in V-shape. It is also known as Verification and Validation model
- **Big Bang Model** – It is a SDLC model where there are no specific processes followed. The development starts with the required money and efforts as the input, and the output is the software developed which may or may not be as per the customer requirement.
- **Agile SDLC model –** This is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product.
- **The RAD (Rapid Application Development) model** – This model is based on prototyping and iterative development with no specific planning. The process of writing the software itself involves the planning required for developing the product.

1.2.1.3 Phases in SDLC and its significance

Software development Life cycle deals with all the phases through which a software will go through, starting from inception of a project to the final maintenance.

Let us take an example of building a house. While building a house ,would we directly build it? For building a house, we need to give our requirements to a construction company or a contractor. While giving the requirements we specify the size of the house,number of rooms required,number of floors we need etc.. Then a meeting is planned between us and the sales person of the construction company.

The requirements that we give are documented and handed over to the architects of the company who then tries to come up with a high level design of the house,which meets our requirements.

We review the high level design shared with us and suggest for any changes. Based on the suggestion given the design is altered. The high level design is shared with us and we review it. We will give back any suggestions that we have, and then based on that the design is altered.

Once the design is finalised,, the architect then develop the final plan/layout of the building. In this final plan, even the grainer level details are specified. For an instance, the information like where to place the TV,what kind of flooring be used etc. would be mentioned.

Eventually,when the plan is finalised and approved,the construction of the house begins. This is the time when the materials required for construction are brought to the site. The first milestone is to complete the foundation. Once the foundation is completed, quality checks are done to ensure that the foundation developed is as per the plan. In that case, we move ahead to the next phase of construction.

Once the construction is completed, necessary inspections are done to check and ensure that all the requirements specified are implemented. We can suggest superficial/smaller changes during this period,which later would be incorporated. this time,which can be. After this, no further changes can be taken up.

Why it's bad when home owners change their minds about the bathroom's location late in a building project.

Once all checks are completed, and all minor changes implemented, the keys are handed over to the user and they can move in. Even after moving in, if there are small problems/issues in the house, the construction company fixes it for the user.

Till now, we discussed about the various steps involved in construction of a house. Similarly a software development also goes through different steps. Developing a software is similar to constructing a house. Both needs a well defined plan. Have you ever thought, what would happen if we do not follow the steps mentioned above in constructing a house? You will start building the house, you will lay the foundation and build the house, half way through constructing the house, you might realise that you want it to be done differently. Think of how much of effort and money will be wasted in demolishing and redoing the house. This is why we make a plan, followed by a design and then start construction.

Similarly, when we are developing a software, we need to capture the requirements first, followed by designing the application. Once these steps are completed, we can go ahead with the actual construction. A typical Software Development life cycle consists of the following stages:

Stage 1: Planning and Requirement Analysis : Requirement analysis is the most important and fundamental stage in SDLC. It is performed by the senior members of the team with inputs from the customer, the sales department, market surveys and domain experts in the industry. This information is then used to plan the basic project approach and to conduct product feasibility study in the economical, operational, and technical areas.

Planning for the quality assurance requirements and identification of the risks associated with the project is also done in the planning stage. The outcome of the technical feasibility study is to define the various technical approaches that can be followed to implement the project successfully with minimum risks.

Stage 2: Defining Requirements : Once the requirement analysis is done the next step is to clearly define and document the product requirements and get them approved from the customer or the market analysts. This is done through 'SRS' – Software Requirement Specification document which consists of all the product requirements to be designed and developed during the project life cycle.

Stage 3: Designing the product architecture : SRS is the reference for product architects to come out with the best architecture for the product to be developed. Based on the requirements specified in SRS, usually more than one design approach for the product architecture is proposed and documented in a DDS - Design Document Specification. This DDS is reviewed by all the important stakeholders and based on various parameters as risk assessment, product robustness, design modularity , budget and time constraints , the best design approach is selected for the product.

A design approach clearly defines all the architectural modules of the product along with its communication and data flow representation with the external and third party modules (if any). The internal design of all the modules of the proposed architecture should be clearly defined with the minutest of the details in DDS.

Stage 4: Building or Developing the Product : In this stage of SDLC the actual development starts and the product is built. The programming code is generated as per DDS during this stage. If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.

Developers have to follow the coding guidelines defined by their organization and programming tools like compilers, interpreters, debuggers etc are used to generate the code. Different high level programming languages such as C, C++, Pascal, Java, and PHP are used for coding. The programming language is chosen with respect to the type of software being developed.

Stage 5: Testing the Product : This stage is usually a subset of all the stages as in the modern SDLC models, the testing activities are mostly involved in all the stages of SDLC. However this stage refers to the testing only stage of the product where products defects are reported, tracked, fixed and retested, until the product reaches the quality standards defined in the SRS.

Stage 6: Deployment in the Market and Maintenance : Once the product is tested and ready

to be deployed it is released formally in the appropriate market. Sometime product deployment happens in stages as per the organizations' business strategy. The product may first be released in a limited segment and tested in the real business environment (UAT- User acceptance testing).
Then based on the feedback, the product may be released as it is or with suggested enhancements in the targeting market segment. After the product is released in the market, its maintenance is done for the existing customer base.

1.2.1.4 What is SRS

Software requirements specification (SRS) is a comprehensive description of the intended purpose and environment for software under development. The SRS fully describes what the software will do and how it will be expected to perform the functionality. An SRS minimizes the time and effort required by developers to achieve desired goals and also minimizes the development cost. A good SRS defines how an application will interact with system hardware, other programs and human users in a wide variety of real-world situations. Parameters such as operating speed, response time, availability, portability, maintainability, security and speed of recovery from adverse events can be evaluated. Methods of defining an SRS are described by the IEEE (Institute of Electrical and Electronics Engineers) specification 830-1998.

SRS describes the behavior of a system to be developed and may include a set of use cases (will be discussed in detail later) that describe interactions the users will have with the software. In addition to, it also contains the non-functional requirements. Non-functional requirements impose constraints on the design or implementation (such as performance engineering requirements, quality standards, or design constraints) .
Software requirements specification establishes the basis for agreement between customers and contractors or suppliers (in market-driven projects, these roles may be played by the marketing and development divisions) on what the software product is to do as well as what it is not expected to do. Software requirements specification permits a rigorous assessment of requirements before design can begin and reduces later redesign.

The following are the benefits of a good SRS

- Establish the basis for agreement between the customers and the suppliers on what the software product is to do
- Provide a good platform which describes the behaviour of the system and also how we are planning to implement the same
- Reduce the development effort
- Provide a basis for estimating costs and schedules
- Provide a baseline for validation and verification.
- Serve as a basis for enhancement.

In projects,Business System Analysts(BSA) who have the better domain knowledge will discuss with the client to get the high level requirements and for clarifications. Then BSA , with other team members will prepare the functional requirements in the SRS. The detailed analysis and design models are prepared by the senior level people within the project.

Please refer to Appendix A for the details that should be captured as part of SRS.

1.2.2 Analysis Phase

1.2.2.1 Importance of Analysis in SDLC

Now that we have the SRS in our hand, what are we going to do next? The next step will be to look at the requirements specified in SRS and try to understand these requirements in detail. This is where analysis phase fits in. Let us now try to understand the various activities that need to be carried out as part of analysis phase.

Analysis is the initial phase in almost all SDLC models. In this phase, the entire scope and functionality of the software application is chalked out with the relevant stakeholders. Stakeholders here mean the user(s) who are going to use the software application.

The main purpose of any application is to solve the problems faced by the people. The most important fact to realize here is that software is built for people, not vice versa . The application must do what the customer wants.

In analysis phase the customer requirements are carefully analyzed to define the functionality of the software to be developed . Here the emphasis is only on what the system will do. The implementation will be discussed in the design stage, which succeeds the analysis phase.

Illustration 1: Software Requirement Specification

Picture 1: User discussion

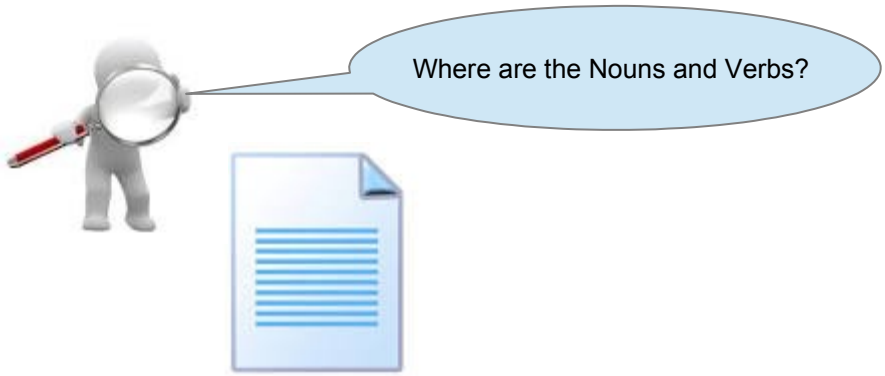
1.2.2.2 Analysis phase in detail - Object Oriented Analysis

Object Oriented principles(OOPs) can be applied during the Analysis phase for flexibility and maintainability.

Why do we need flexibility & maintainability in our code?

If our code is more maintainable and flexible, we would be able to incorporate changes into the code quite easily. This will help us in adapting quickly to changes in customer needs.

In case the customer wants to add more functionality to an existing application, you should be able to accommodate them with minimal changes. This can be made possible to if OOPs principles are applied during Analysis phase.



The various steps in object oriented analysis are as follows :-

1. Read the problem description carefully.
2. Identify the nouns and the verbs associated with each noun.
3. Each noun can be thought of as an object, having properties or attributes. The values of the attributes of object is said to define the state of the object.
4. The relationship between these newly formed objects need to be analyzed. Object can have behaviour as described by the verbs associated with it.

Abstraction is the technique of deciding what objects are required as well as deciding which attributes(state) are put in these objects.

For eg:- Consider an online car rental system which enables customer to book a cab from a source to the destination. The customer should be able to check for availability for a particular make (or any other preferences) and make a booking in the system. Admin should be able to add more cabs. System should also be able to process the bill for the customer and calculate the gross rent amount.

Identify what could be the various objects and their attributes. Also identify the relationships between the these objects.

Considering the nouns, the objects may be

1. Cab

2. Customer
3. Bill
4. Admin
5. Car Rental System
6. Booking etc

We can also identify relations between these objects via verbs.
Eg:-
Customer **books** cab.
System **processes** bills as per the **booking**.
Admin **adds** cab.
etc

1.2.2.3 UML and its significance during Analysis

Unified Modeling language(UML), is a visual modeling language to document software architecture and business process.
It is a set of graphical notations and diagrams that help in specifying all the aspects of any system. UML essentially captures the behavioral and structural aspects of the system under study. It may also contain a brief documentation in a specific template.

The current version of UML is UML 2.4.1.

The following are the types of UML diagrams normally used in documenting systems :-

1. Use case diagrams
2. Domain Object Model Diagram
3. Sequence Diagram
4. Class Diagram

Each UML diagram has its own purpose. A single type of UML diagram will not give a complete picture of the system. To represent a system in its entirety, a combination of UML diagrams mentioned above maybe required.

For eg - Use case diagram is a high level diagram that captures the requirements of a system from the user's perspective. It represents the system functionality. It emphasizes on “what” the system has to do.

Sequence Diagram, on the other hand, emphasizes on “how” the functionality is implemented in the system. It represents “how” the information is passed between the various components of the system like User Interface, Business Logic and the Database.

During the analysis phase, the user's interaction with the system is analysed at a highest level possible. Also, we know that the goal of Object Oriented analysis is to identify the objects in the

problem description and its relationships with each other. These findings are documented in Usecase model and domain object model diagram. We shall look into them in detail.

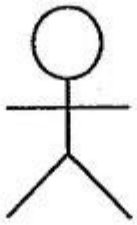
1.2.2.4 Usecase Model

1.2.2.4.1 Usecase diagram

The use case diagram represents a user's interaction with the system.
The user is known as an actor here. Actors are differentiated from each other on basis of their role with respect to the system.
Use cases should start off at the highest level possible. Also it should be structured from the actor's perspective. Going forward, we shall be referring to the user as an actor.

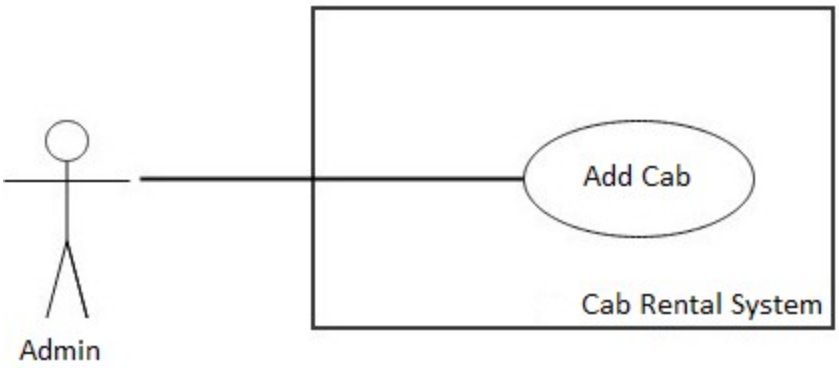
1.2.2.4.2 UML Notations in Use case diagram

Actors
A use case diagram specifies the interaction between the system and external entities. These entities are referred to as actors. An actor is usually drawn as a named stick figure, or as a rectangle with the «actor» keyword.

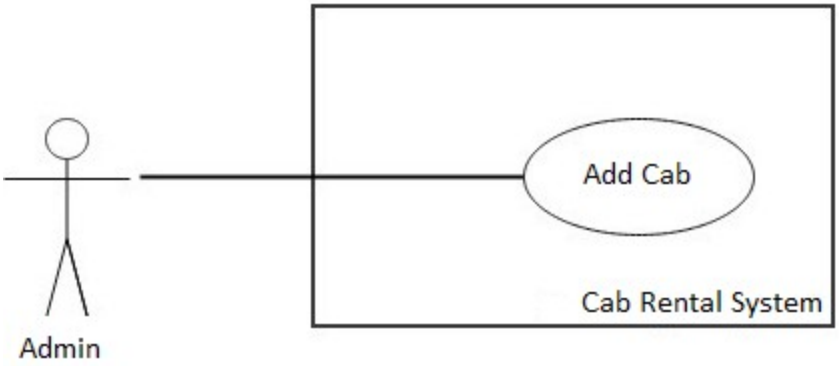


Actor

Use Cases
A use case provides a high-level view of behavior observable outside the system. The notation for a use case is an ellipse.
The notation of a use case is complete when the actor is connected to the use case via a connecting line. This connecting line can have an optional arrowhead showing the direction of control.



System Boundary
Usually use cases are depicted as being inside the system and actors as being outside the system. Consider the following use case diagram that shows the admin adding cab to the cab rental system.



1.2.2.4.3 Writing Use cases

Use case mainly consists of the following -

i) Use Case Title - It mainly describes the goal of the use-case.
ii) Scope – The name of the system in which the use case is observed.
iii) Level – The level at which the use case has been written. Can be User Goal (use cases which are directly used by the user) or Subfunction(use cases which are invoked as part of another use case eg:- cab Id generation while adding a cab).
iv) Primary Actor – The actor who is invoking the use case.

v) Stakeholders & Interests - Anyone who is affected by this use-case and in what way they will be affecetd.
vi) Precondition – The conditions that must be met before the use case can start.
vii) Success guarantee - The conditions that must be met for a valid end of the use case.
viii) Main success scenario - The main part of a use case is its scenario. A scenario describes a sequence of steps that are executed in order to meet the objective of the use case. It denotes the successful completion of the use case i.e the events turned out exactly as planned, hence it is known as the main success scenario.
ix) Extensions: Using Extensions we can specify how the system should behave in case of foreseen or unforeseen errors. This is where we can specify the alternate routes with respect to the step specified in main success scenario.

Use cases must be written in active voice. The sequence of steps given should be as detailed as possible, leaving no space for assumption.

eg:-
The following is the use case for the admin actor to add a cab to the system.

Use Case UC1:	Add Cab
Scope:	Cab Rental System
Level:	Admin goal
Primary Actor:	Admin
Stakeholders and interests:	• Admin: Wants to add cab to the system • Car Rental System: Wants to make process of adding cab fast and reliable.
Preconditions:	• Admin should be logged in.
Success Guarantee:	• Cab details given by the Admin is recorded. • Cab Id is generated.
Main Success Scenario:	1. Admin views the Home page of Cab Rental System. 2. Admin clicks on add cab link. 3. System displays registration page. 4. Admin enters the following details: ⅄ Cab name, ⅄ Cab make, ⅄ Name of the driver. 5. System validates the following details: ⅄ Cab name should not be a number or null and of maximum 15 characters, ⅄ Cab make should not be a number or null and of maximum 15 characters

	⅄ Driver name should not be a number or null and of maximum 15 characters. 6. System generates a confirmation message. 7. Admin confirms the registration. 8. System generates cab Id for the cab. 9. System saves cab details. 10. Admin gets the cab Id and details with confirmation message.
Extensions:	5a. Admin enters invalid details ⅄ Cab name is null or number or more than 15 characters. ⅄ Cab make is null or number or more than 15 characters. ⅄ Driver name is null or number or more than 15 characters. 8a. Cab with same name, make and driver name already exists. ⅄ System displays an error message, cab already exists. ⅄ System displays main menu. 9a. System fails to save the cab details provided because server is down. ⅄ System displays an error message. ⅄ System displays main menu.

If you observe closely, extensions are named from the steps in the main success scenario from where it can be derived.

1.2.2.5 Domain Model Diagram

The domain model describes how the entities from the use cases cooperate in the real-life world of the system domain. It specifies the various entities, their properties, relationships with each the. It helps us in limiting the scope of the system domain. The understanding of the system domain can be reviewed and communicated amongst the stakeholders. It helps the stakeholders to focus if used effectively as a communication tool.

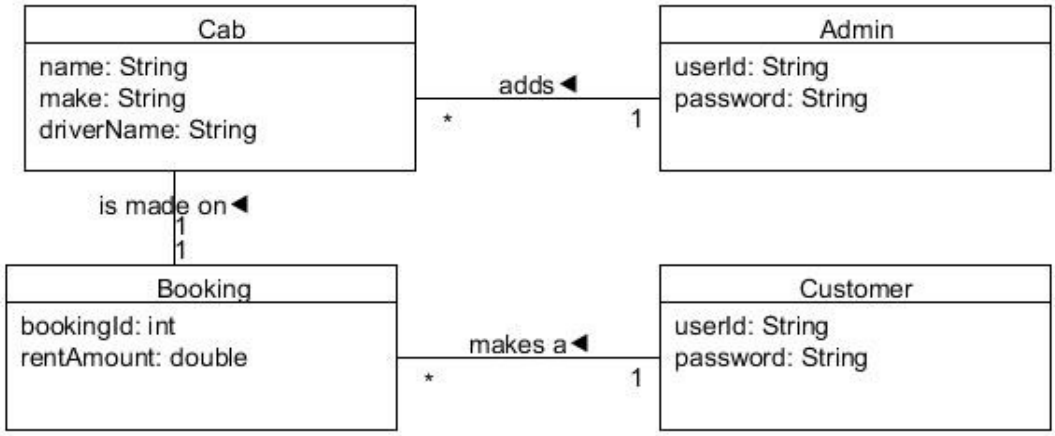
Features of a domain model

• **Domain classes** – each domain class denotes a type of object identified from the problem description. Each object is defined in a rectangular box, having two compartments. The first compartment holds the object name. The second compartment contains the list of attributes/properties.

• **Attributes** – an attribute is the description of a named slot of a specified type in a domain class; each instance of the class separately holds a value.

• **Associations** – an association is a relationship between two (or more) domain classes that describes links between their object instances. Associations can have roles, describing the multiplicity and participation of a class in the relationship.

• **Additional rules** – complex rules that cannot be shown with symbolically can be shown with attached notes.
The following diagram shows the domain model diagram for the cab rental system.



1.2.2.6 Deliverable at the end of Analysis phase

The deliverable at the end of Analysis phase are mainly the following UML artifacts:-

I) Use Case model – consisting of use cases and use case diagrams
II) Domain Model Object Diagram

The Analysis phase aims in specifying what the system does, at the highest level. All assumptions about the usage of the system under study are documented here. The scope is chalked out without divulging the implementation details.

1.2.2.7 Summary

Analysis is a process of discovery where the development team models and understands the requirements of the system.
The Analysis phase aims in specifying what the system does, at the highest level. All assumptions about the usage of the system under study are documented here. The scope is chalked out without divulging the implementation details.

1.2.2.8 Assignment

Consider automating a fast food chain outlet. Customers places order by picking items from the menu. The employee at the counter enters the order details in the system. Items can be ready made or customized. For example, the customer can either go for a veg burger which comes with standard ingredients or customize their veg burger by giving their options, for example – no onions, extra cheese and olives. Once customer finishes placing the order, the employee at the counter generates the bill and assign a token number for the customer. Status of the order at this time is "PLACED". At the time of generating the bill, discount coupons can be applied, if any. The order is then passed on to the order processing team, who will then put the items in a try as per the order. Once the order is picked by the order processing team, the status is changed to "PROCESSING". Once all items in the order are placed in the tray, the status of the order is changed to "COMPLETE". The order is then picked up by the delivery team and the order is delivered at the customers table.

From the scenario given above:

1. Identify the actors
2. Draw the use case diagram
3. Write the detailed use cases
4. Identify the objects involved and their relationship and come up with the domain level class diagram.

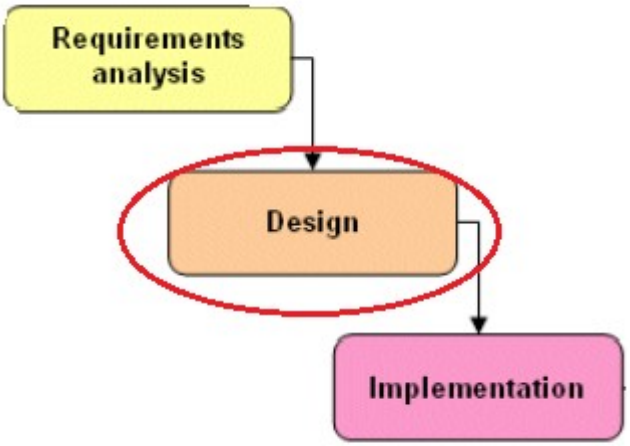
NOTE: Try out drawing use case diagram and domain level class diagram using Violet tool. It is very easy and self explanatory to use it. Same is available in Violet folder along with content of current topic. Alternately, you can also explore using eclipse to draw these diagrams. Refer document **1_Content-b UserManual_UML** available in same folder.

1.2.3 Design Phase

1.2.3.1 Importance of Design in SDLC

We are using the outcome/results of analysis phase as inputs in the

Design phase. During the design phase,the designer's will develop the design model which can be used for the developers to implement the code. The emphasis is on making a practical,working design for what has been outlined in the requirements analysis phase. This phase is very important because it describes what is going to get developed in the next phase. Also poor design can lead to the failure of projects.



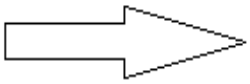
1.2.3.2 High Level Design

High Level Design (**HLD**) is the overall system design which covers the system architecture and the database design. It describes the relation between various modules and functions of the system. A high-level design document will usually include a high-level architecture diagram depicting the components, interfaces and networks that need to be further specified or developed. The document may also depict or otherwise refer to work flows and/or data flows between component systems.

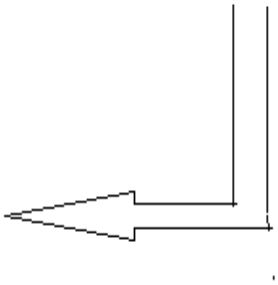
To minimize the maintenance overhead as construction proceeds and the lower level is done it is best that high level design is only elaborated to the degree needed to satisfy these needs. A **high-level design** provides an overview of a solution, platform, system, product, service, or process. Such an overview is important in a multi-project development to make sure that each supporting component design will be compatible with its neighboring designs and with the big picture. Today, most high-level designs require contributions from a number of experts, representing many distinct professional disciplines. Thus **High-level design** provides an overview of an entire system, identifying all its elements at some level of abstraction.

Consider a scenario in which a family wants to construct their house . Firstly they need to buy/have a plot for construction. Then a detailed analysis has to be done w.r.t the structure,floors and rooms . . Later the engineer/architect will design the house as per the customer request.. After the approval from the client, they will provide the detailed plan for the building . Similar,ly we need to design /choose ***an architecture*** *that will be more suitable to the specific application/project. That is what it would happen in the High Level Design (HLD) phase.*

Users discuss the requirements with the building architects



As per the requirement ,the architects is prepares the high level design/model



High level design/model

A HLD is referred to as **software architecture. Software** architecture serves as the blueprint for both the system and the project developing it, defining the work assignments that must be carried out by design and implementation teams.

1.2.3.2.1 Different types of Software Architecture

Architecture is an artifact for early analysis to make sure that a design approach will yield an acceptable system. By building effective architecture, you can identify design risks and mitigate them early in the development process. The architecture of a system is a comprehensive framework that describes its form and structure -- its components and how they fit together. The term “software architecture” is used to denote three concepts:

- High level structure of a software system
- Discipline of creating such a high level structure
- Documentation of this high level structure

The term ***software architecture*** was first used in the late 1960s , but became prevalent only in the beginning of the 1990s. The field of computer science had encountered problems associated with complexity since its formation. Earlier problems of complexity were solved by developers by choosing the right data structures developing algorithm , and by applying the concept of separate concerns. Although the term “software architecture” is relatively new to the industry, the fundamental principles of the field have been applied sporadically by software engineering pioneers since the mid-1980s. The architecture should:

- Expose the structure of the system but hide the implementation details.
- Realize all of the use cases and scenarios.
- Try to address the requirements of various stakeholders.
- Handle both functional and quality requirements

An **architectural style**, sometimes called an architectural pattern, is a set of principles—a coarse grained pattern that provides an abstract framework for a family of systems. An architectural style improves partitioning and promotes design reuse by providing solutions to frequently recurring problems.

Category	Architecture styles
Communication	Service-Oriented Architecture (SOA), Message Bus
Deployment	Client/Server, N-Tier, 3-Tier
Domain	Domain Driven Design
Structure	Component-Based, Object-Oriented, Layered Architecture

The following are the different architectures:

Architecture style	Description
Client/Server	Segregates the system into two applications, where the client makes requests to the server. In many cases, the server is a database with application logic represented as stored procedures.
Component-Based Architecture	Decomposes application design into reusable functional or logical components that expose well-defined communication interfaces
Domain Driven Design	An object-oriented architectural style focused on modeling a business domain and defining business objects based on entities within the business domain
Layered Architecture	Partitions the concerns of the application into stacked groups (layers).
Message Bus	An architecture style that prescribes use of a software system that can receive and send messages using one or more communication channels, so that applications can interact without needing to know specific details about each other.
N-Tier / 3-Tier	Segregates functionality into separate segments in much the same way as the layered style, but with each segment being a tier located on a physically separate computer
Object-Oriented	A design paradigm based on division of responsibilities for an application or system into individual reusable and self-sufficient objects, each containing the data and the behavior relevant to the object.
Service-Oriented Architecture (SOA)	Refers to applications that expose and consume functionality as a service using contracts and messages

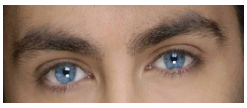
As you know **N-tier** just refers to the physical structure of an implementation whereas **MVC** is a programming design pattern where different portions of code are responsible for representing the Model, View, and controller in some application.
Conceptually the three-tier architecture is linear. This means client tier never communicates directly with the data tier,it will be only through the middle tier .But MVC architecture is triangular. The MVC design pattern will be discussed in detail in the next section.

1.2.3.2.2 MVC Architecture

Model-View-Controller (MVC) is a **fundamental design pattern** for the separation of user interface logic from business logic .This architecture has been widely adopted as an architecture for World Wide Web (WWW) applications in major programming languages. A MVC pattern is made up of the following three parts:- Model ,View & Controller.

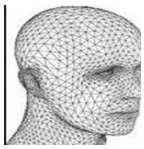
Consider the following real life example for getting the idea regarding the MVC architecture.'*John saw his old friend and smiled*.'
John sees his friend through his eyes. Then through eyes the information reaches the brain. And the brain identifies the person and shows smile to his friend. Using face's **neural network**,brain will make you to smile.

- **neural system** of a body stands for **Model(M)**
- **eyes** of a human stand for **View(V)**
- **brain** stands for **Controller(C)**



Seeing through his eyes

information reaches the brain and call the respective action

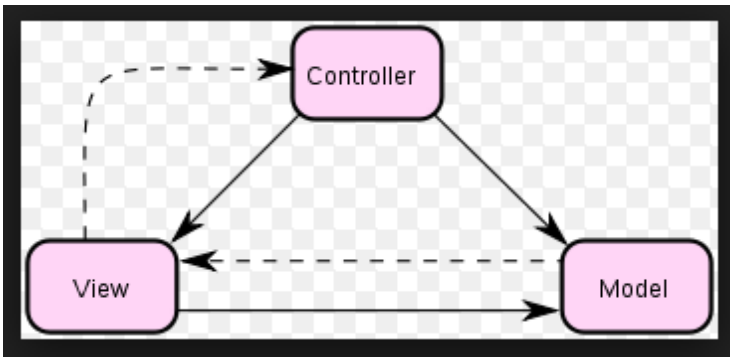


neural n/w of face is involved in doing the action

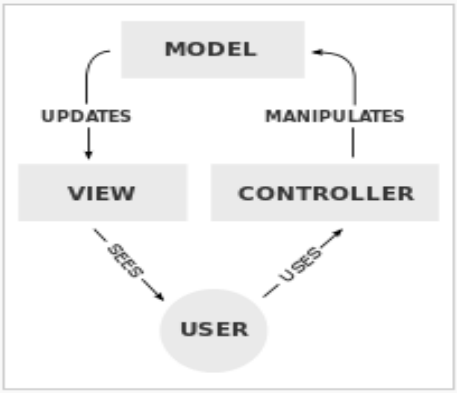
response is shown using the lips.

- **Model** - Is responsible for maintaining data/state and business logic(include bean and service classes)
- **View** – Is responsible for displaying all or a portion of the data to the user.(include html/jsp)
- **Controller** - Software Code that controls the interactions between the Model and View. (include servlet which decide the action according to 'GET' or 'POST' method)

MVC Architecture has implemented by Trygve Reenskaug at 1979 for the first time. It was implemented on Smalltalk at Xerox labs. The benefits and advantages of the architecture has been accepted by most of the coders and software engineers.



MVC is popular as it isolates the application logic from the user interface layer and supports separation of concerns. Here the Controller receives all requests for the application and then works with the Model to prepare any data needed by the View. The View then uses the data prepared by the Controller to generate a final presentable response.

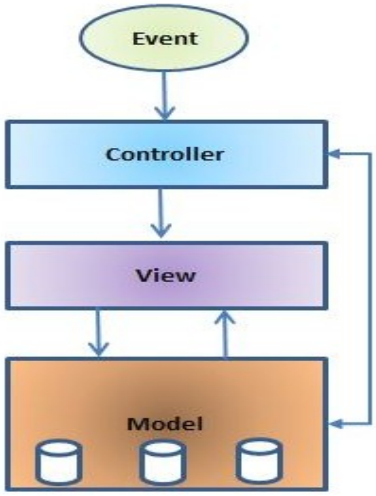


A typical collaboration of the MVC components

As discussed,

- A controller can send commands to the model to update the model's state (e.g., editing a document). It can also send commands to its associated view to change the view's presentation of the model (e.g., by scrolling through a document).
- A model notifies its associated views and controllers when there has been a change in its state. This notification allows the views to produce updated output, and the controllers to change the available set of commands. In some cases an MVC implementation might instead be "passive," so that other components must poll he model for updates rather than being notified.
- A view requests information from the model that it needs for generating an output representation to the user

Benefits of MVC pattern

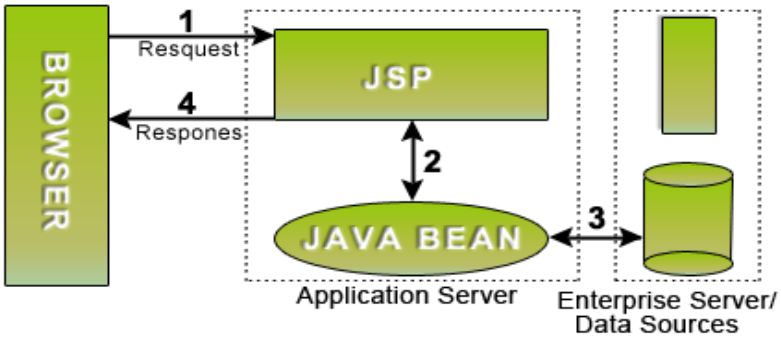


1. Separation of components:
   The separation the three components(model,view and controller), allows the re-use of the business logic across applications. Multiple User Interfaces can be developed without concerning the code base
2. Developer specialization and focus:
   The developers of UI can focus exclusively on the UI screens without bogged down with business logic. The developer of Model / business can focus exclusively on the business logic implementations, modifications, updates without concerning the look and feel and it has nothing to with business logic.
3. Parallel development by separate teams:
   Business logic developers can build the classes, while the UI developers can involve in designing UI screens simultaneously, resulting the interdependency issues and time conservation. UI updates can be made without slowing down the business logic process. Business logic rules changes are very less that needs the revision / updates of the UI

1.2.3.2.3 MVC architecture Models

In **Model 1 architecture t**he request is processed by JSP or Servlet. The JSP or Servlet handles all the responsibility of request processing such as validating the data, business logic validation and processing, show error messages, finally generating the view for the application.

JSP Model 1 architecture is good for very small application, but it's not a good solution for big enterprise application. The JSP Model 1 architecture do not separate the view, business processing logic and the view is tightly integrated with the business logics.



Model1 Architecture

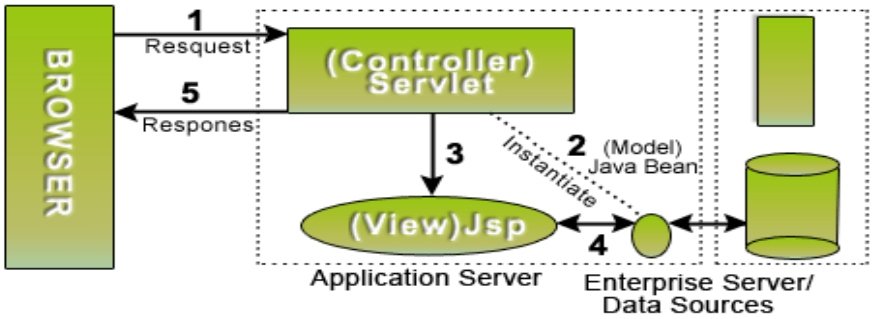
The above figure clearly shows the flow of the model1 architecture.

- Browser sends request for the JSP page
- JSP accesses Java Bean and invokes business logic
- Java Bean connects to the database and get/save data
- Response is sent to the browser which is generated by JSP
- Advantage of Model 1 Architecture
  •Easy and Quick to develop web application

Disadvantage of Model 1 Architecture

•Navigation control is decentralized :Since every page contains the logic to determine the next page. If JSP page name is changed that is referred by other pages, we need to change it in all the pages that leads to the maintenance problem.
•
•Time consuming :We need to spend more time to develop custom tags in JSP. So that we don't need to use scriptlet tag.
•
•Hard to extend:It is better for small applications but not for large applications.

Model 2 architecture is based on the popular MVC architecture. Here Model, View and Controller is responsible for the specific work for making the architecture ideal for developing complex enterprise web applications.
In JSP Model 2 architecture, JSP is used for creating the view for the application. A centralized Servlet is used to handle all the request for the application. The Servlet works as the controller for the application. It then uses the Java beans for processing the business logic and getting the data (Model) from the database. Finally it uses the JSP to render the view which is displayed to the user.



Model2 Architecture

The above figure clearly shows the flow of the model2 architecture.
In this , a central servlet, known as the Controller, receives all requests for the application. The Controller then processes the request and works with the Model to prepare any data needed by the View, and forwards the data to a JSP. The JSP then uses the data prepared by the Controller to generate a response to the browser. In this architecture, the business and presentation logic are separated from each other. Having the separation of business and presentation code accommodates multiple interfaces to the application, this separation provides excellent reuse of code.

Advantages of Model 2 (MVC) Architecture

- Navigation control is centralized.Now only controller contains the logic to determine the next page.
- Easy to maintain,extend and test.
-
- Better separation of concerns

1.2.3.3 Introduction to DAO Layer

DAO is referred to as Data Access Object .As the name implies,this layer is mainly
used to provide the interactions with the database(ie data access) using JDBC. For every CRUD (Create-Read-Update-Delete) operations,we need to interact with the database according to business logic. Thus it deals with all the queries that we need to perform using JDBC.

The Data Access Object (DAO) is now a widely accepted mechanism to abstract away the details of persistence in an application. The idea is that instead of having the domain logic communicate directly with the database, file system, web service, or whatever persistence mechanism your application uses, the domain logic speaks to a DAO layer instead. This DAO layer then communicates with the underlying persistence system or service.

The advantage of the DAO layer is that if you need to change the underlying persistence mechanism you only have to change the DAO layer, and not all the places in the domain logic where the DAO layer is used from. The DAO layer usually consists of a smaller set of classes, than the number of domain logic classes that uses it. Should you need to change what happens behind the scene in the DAO layer, the operation is somewhat smaller, since it only affects the DAO layer. It is also a somewhat more controlled operation, since you can search for all DAO classes, and make sure they are changed to use the new persistence mechanism.

For example If we want to insert the details to the database from the form page(UI) and want to show the success/failure message accordingly. Then according to the MVC pattern,we want to capture the form element values to the controller .Then need to set the values to the respective bean and the same will get transferred to the service layer .From the service layer it will get transferred to the dao layer (eg:ProductDao.java) where the actual database interaction is happening using JDBC. Also we used to create the separate util class (DatabaseUtil.java) which will maintain the connectivity statements needed for JDBC. Thus the data will get inserted into the database.
The below representation is used to demonstrate the insert operation as discussed above.

User Interface

|

Controller ------ Bean

|

Service

|

DAO ----- DatabaseUtil

The sample dao class and util class is given below.

ProductDao.java

```java
public class ProductDao {
    Connection conn = null;
    PreparedStatement pst = null;
    ResultSet rs = null;

    public boolean insertDetailsByBean(ProductBean bean) {
        boolean flag = false;
        try {

            conn = DatabaseUtil.getConnection();

            String sql = "insert into product values(prod_seq.nextval,?,?,?)";
            pst = conn.prepareStatement(sql);
            pst.setString(1, bean.getProductName());
            pst.setString(2, bean.getProductDesc());
            pst.setString(3, bean.getStatus());
            int i = pst.executeUpdate();
            if (i > 0) {
                flag = true;
            }
        } catch (SQLException e) {
            System.out.println(e);
        } catch (Exception e1) {
            System.out.println(e1);
        } finally {
            DatabaseUtil.closeConnection(conn);
            DatabaseUtil.closeStatement(pst);
        }
        return flag;
    }
```

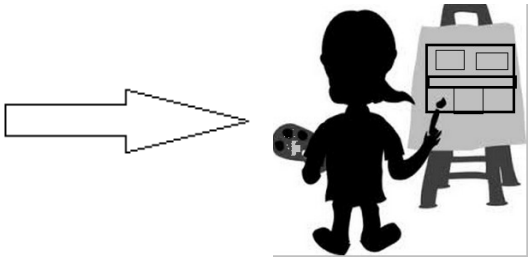
1.2.3.4 Low Level Design

Low Level Design (LLD) is like detailing the high level design(HLD). A low-level description is one that describes individual components, provides detail rather than overview, rudimentary functions rather than complex overall ones, and is typically more concerned with individual components within the system and how they operate. It defines the actual logic for each and every component of the system.
Class diagrams with all the methods and relation between classes comes under LLD. A good Low Level Design Document developed will make programming very easy for developers. Code will be developed by developers by directly referring to the LLD. If the requirements are analyzed correctly during the analysis phase, then code developed from Low Level Design will require minimal effort for debugging and testing.

As per the earlier scenario which we discussed during HLD,the architect will make the low level design or the plan of the building and would clearly illustrate the structure/location of each room with all the proper dimensions.

.

Family discusses their requirements with the building architect



As per the requirement ,the architect prepares the high level design/model

low level design/plan

high level design/model with specific architecture style

In the software industry,the designers/senior level people will make the sequence diagram (according to the design pattern/architecture that we have selected) and the class diagram as per the high level design. The sequence diagrams and the class diagrams will be explained in the following sections.

1.2.3.4.1 Significance of UML in Low Level Design

In low level design focus is given on what needs to be done. This is the time to start visualizing the application. Here we also need to focus on the way we need to implement each of the functionality mentioned in the user requirement. It will be easier to represent this using diagrams than using plain text. Here comes the significance of UML in Low Level Design. UML helps in giving an overall picture of what needs to be done in the code level, for each of the use case that is documented. The main techniques used in UML that we will be focusing on to capture the Low Level design are : Sequence Diagram and Design Level Class Diagram. Let us now discuss these in detail.

1.2.3.4.2 Sequence Diagram in detail

A **sequence diagram** is an interaction diagram that shows how operations are carried out. It also shows what messages are sent and when. A sequence diagram shows object interactions arranged in a time sequence. It depicts the objects and classes involved in the scenario. It also illustrates the sequence in which messages are exchanged between the objects for fulfilling a functionality. Sequence diagrams are logically connected to the use cases.

To explain in simpler terms, this is the time to start visualizing the application and to think about how the code will be organized. We also need to think about the flow of information within the application. In object oriented world, all tasks are handled by objects. It essentially means that we need to identify the objects that are needed to complete the task, and the sequence(order) in which the objects should be invoked.

The major input for finding out the objects will be the Domain model object diagram which was prepared as part of analysis phase.

A sequence diagram shows, as parallel vertical lines (*lifelines*), different processes or objects that live simultaneously, and, as horizontal arrows, the messages exchanged between them, in the order in which they occur.

A sequence diagram shows:

- different processes or objects that live simultaneously, as parallel vertical lines (*lifelines*).
- the messages exchanged between objects, sequentially, as horizontal arrows.

Importance of Sequence Diagram

- Depict object interactions for a given Use Case

- Specify the messages passed between objects using horizontal arrows, including messages to/from external actors
- Time progresses from Top to bottom

Key parts of a sequence diagram.

participant: The objects that are required in fulfilling a requirement specified in a use case. These objects can be identified by picking the nouns from the use case scenario. These objects are already present in the domain model object diagram.

Sequence diagram starts with an unattached "found message" arrow which represents the start of an user initiated event.

message: Information passed between participant objects inorder to fulfill the requirement specified in the use case. This is represented using horizontal arrows in the sequence diagram.

The axes in a sequence diagram:

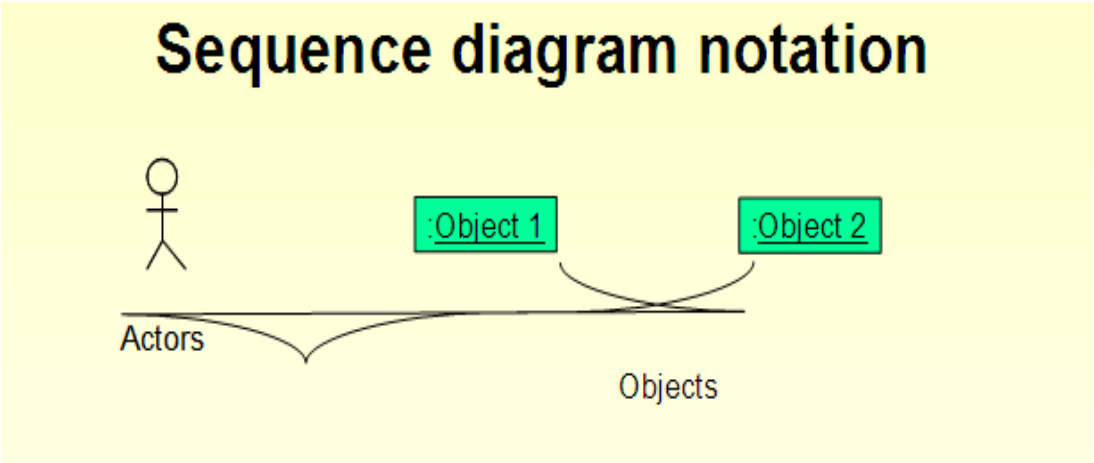
horizontal axis: Specifies which objects/participants is acting and the information passed between them.
Vertical axis: Specifies time, time progresses from top of the diagram to the bottom.

1.2.3.4.3 Creating sequence diagrams

There are a few steps involved in creating sequence diagram. They are:

- Identify the relevant objects involved in the use case.
- Establish the role of each object.
- Identify the controller.
- Decide on the messages between the objects.

Now, let us have a look at the various UML notations used in developing a sequence diagram.

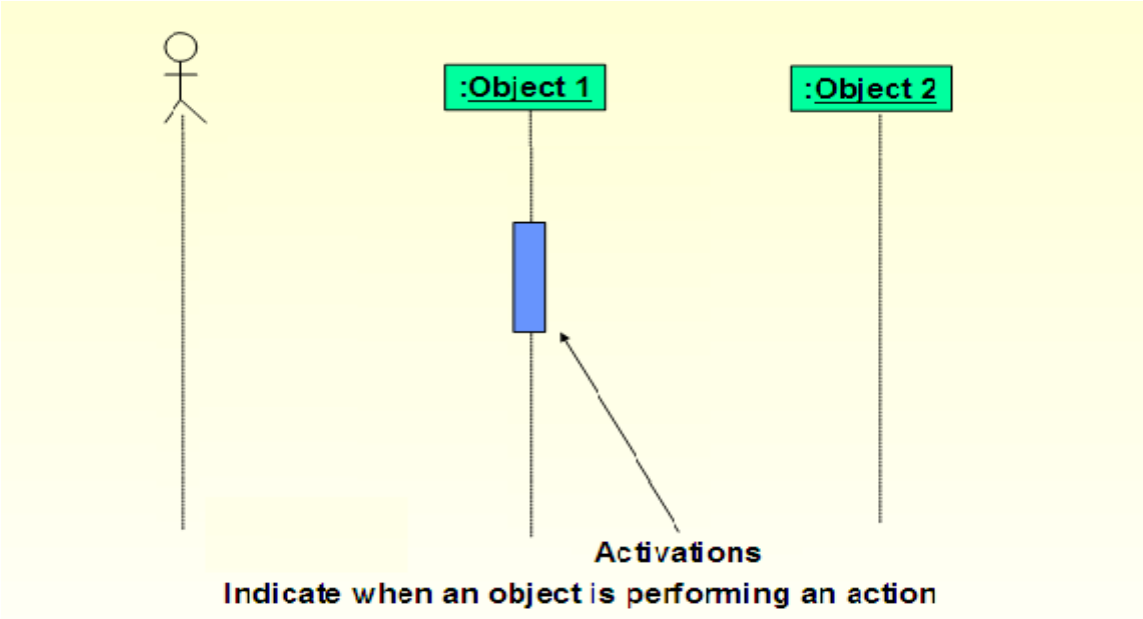


Picture 1: Different notations used in sequence diagram

In the diagram given above, the actor involved in the use case is represented by the stick figure. Object1 and Object2 are the participant objects.

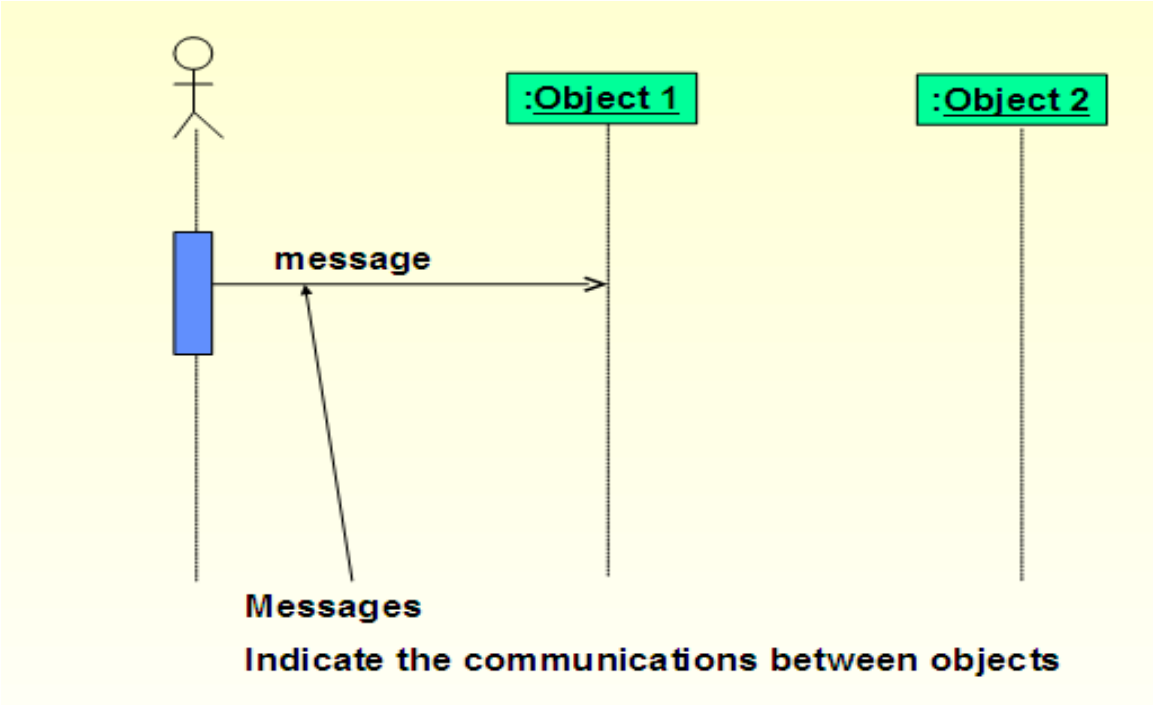
To denote the existence of an object, life lines are used in a sequence diagram. These are represented by vertical dotted lines originating from the participant object. This is illustrated in the diagram given above.

When we are defining a sequence, it is important to show when the object is activated. The object is activated when a method is invoked on that object. It remains active till the entire method is executed. This active duration of the object is denoted using a vertical rectangular bar in the life line of the participant object. This is depicted in the diagram given below.

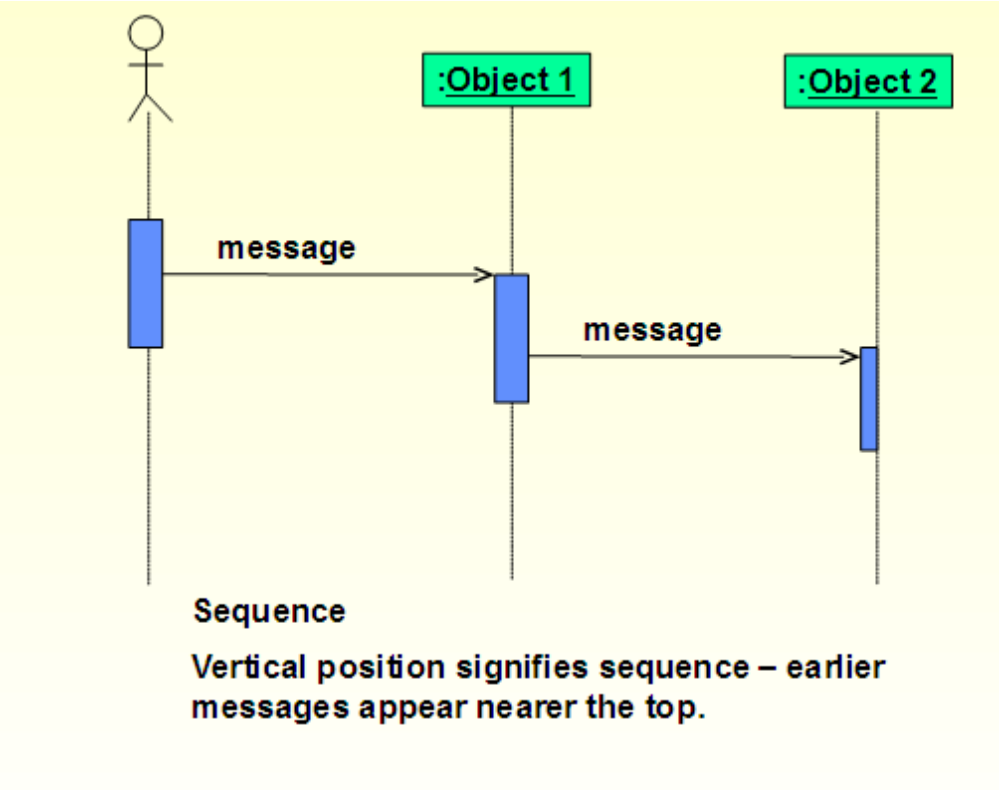


Picture 2: Sequence diagram depicting activation of an object

In object oriented world, objects need to communicate with each other to perform a task. This is done by invoking methods with required parameters(or messages).



Picture 3: Communication between objects

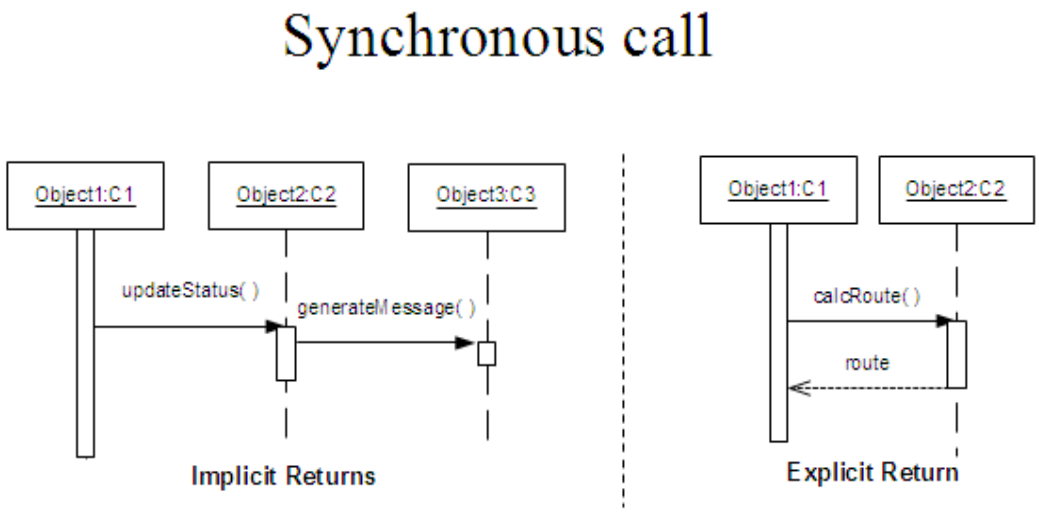


Picture 4: Time progression in sequence diagram

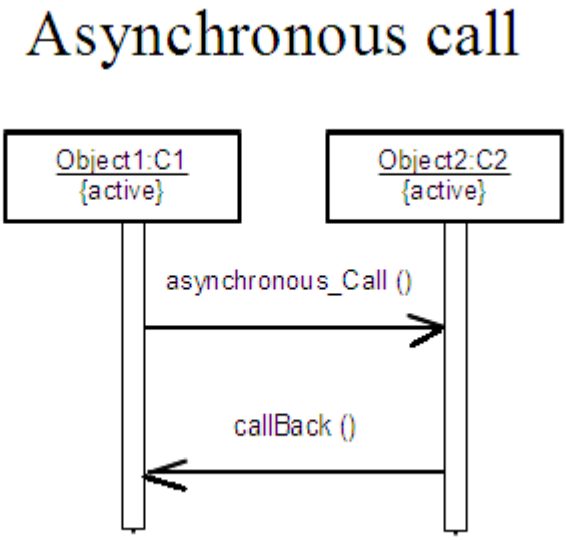
The message passing between two objects can be represented in following ways :

Type	Graphic	Description	Representation
Asynchronous		A line with an open arrowhead	This graphic represents an asynchronous signal or an asynchronous call in which the source object sends the message and immediately continues with the next step.
Synchronous		A line with a solid arrowhead that points toward the receiving lifeline	This graphic represents a synchronous call operation in which the source object sends a message and waits for a return message from the target before the source can continue.
Synchronous return		A dashed line with a solid arrowhead that points toward the originating lifeline	This graphic represents a return message from a call to a procedure. When you create a synchronous message, a return message is created by default.

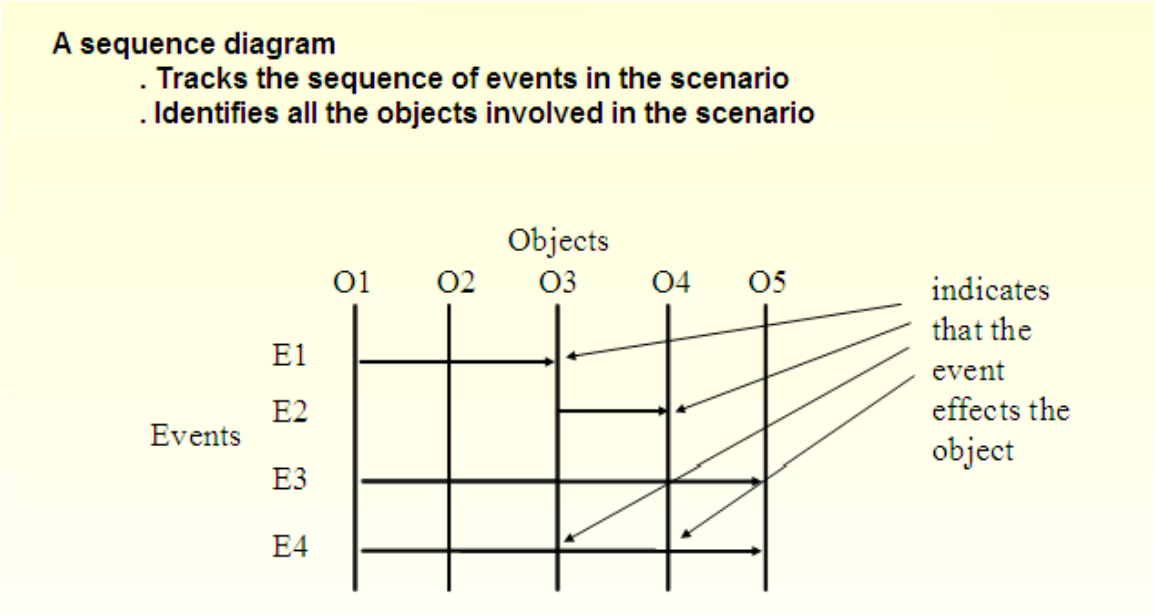
Picture 5: Types of messages



Picture 6: UML representation of Synchronous call

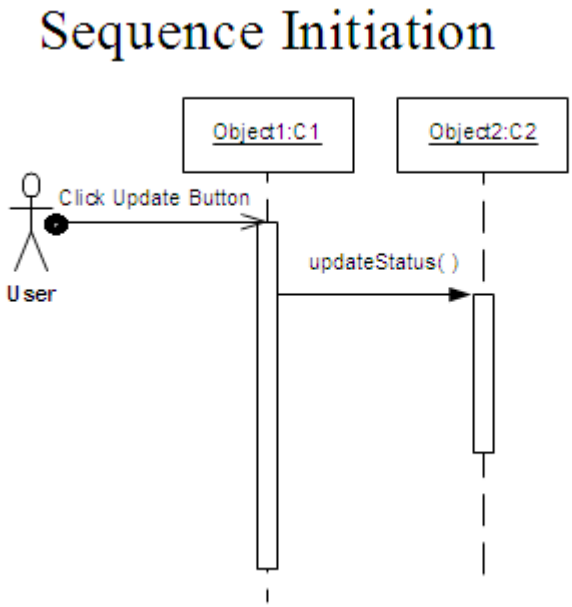


Picture 7: UML representation of asynchronous call



Picture 8: Sequence diagram

The sequence is usually initiated by an actor. This is the starting point of the sequence. UML representation of sequence initiation is given below. The initiation of sequence is represented using an arrow with a filled circle at one end.

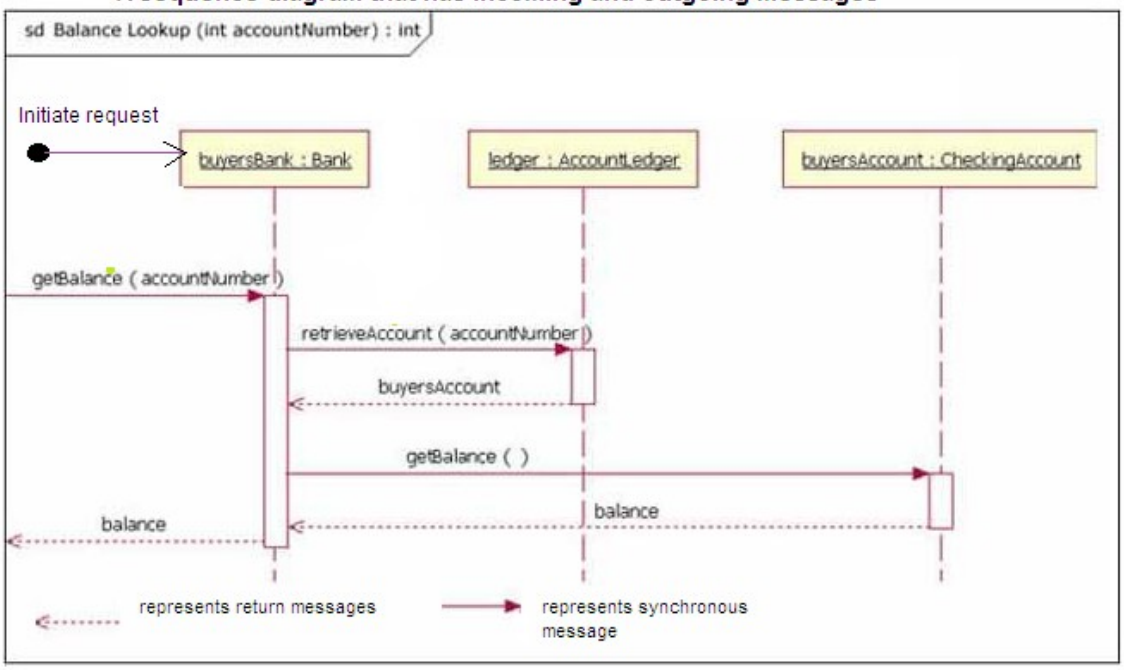


Picture 9: UML representation of sequence initiation

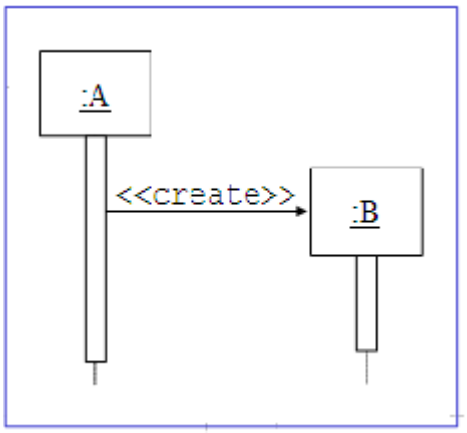
The diagram given below depicts the UML notations that we have discussed so far. The scenario under consideration is the account balance checking in a bank. Following are the steps involved in checking the account balance.

1. Actor initiates the balance checking event.
2. System prompts the actor to enter the account number
3. User enters the account number.
4. System retrieves the account details from Account Ledger.
5. System returns the balance amount in the account.

A sequence diagram that has incoming and outgoing messages



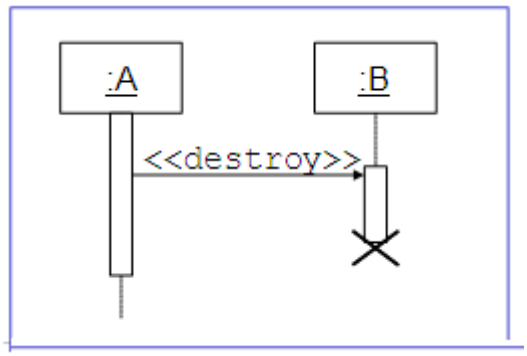
When we are developing the sequence diagram, it will be required to indicate that an object is created. Using UML notations, object creation is represented as given below.



Picture 10: Object creation

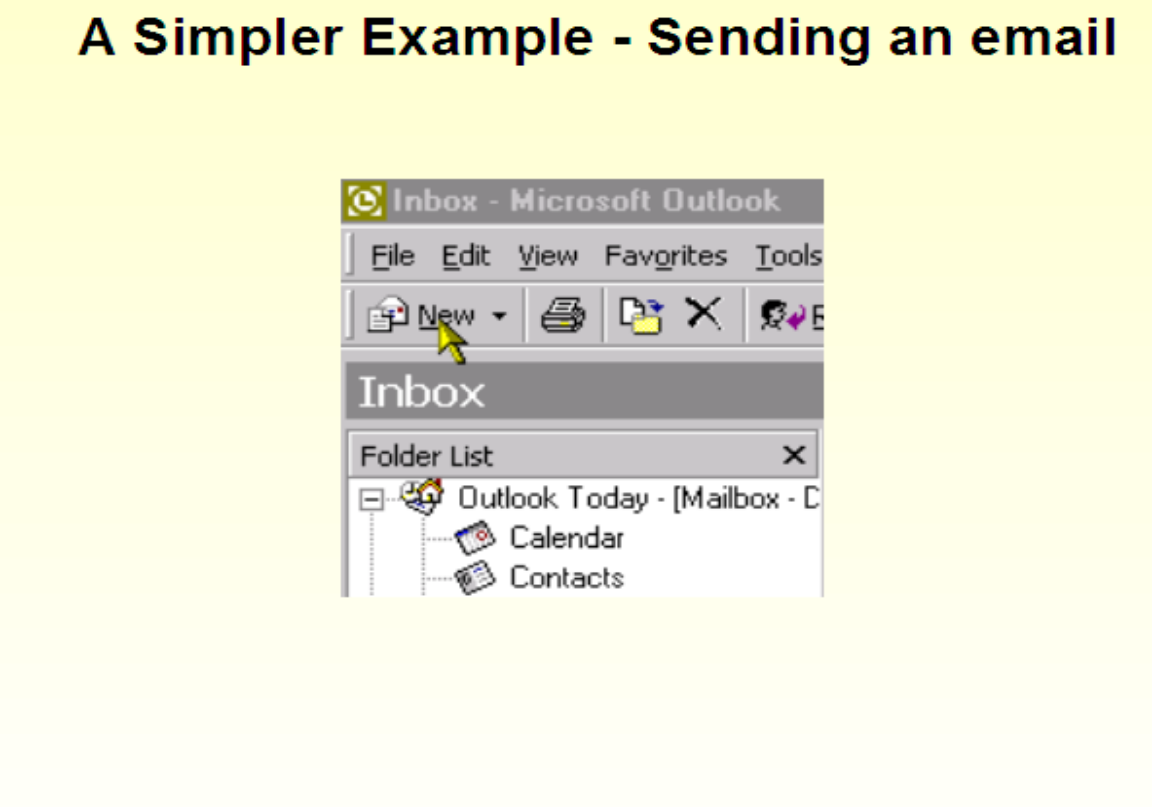
Object Destruction

Similar to object creation, it might be required to destroy an object. In UML notation, destroying an object is represented using <<destroy>> message. Destroy messages are optional in a sequence diagram. Avoid including object destruction unless memory management is critical.



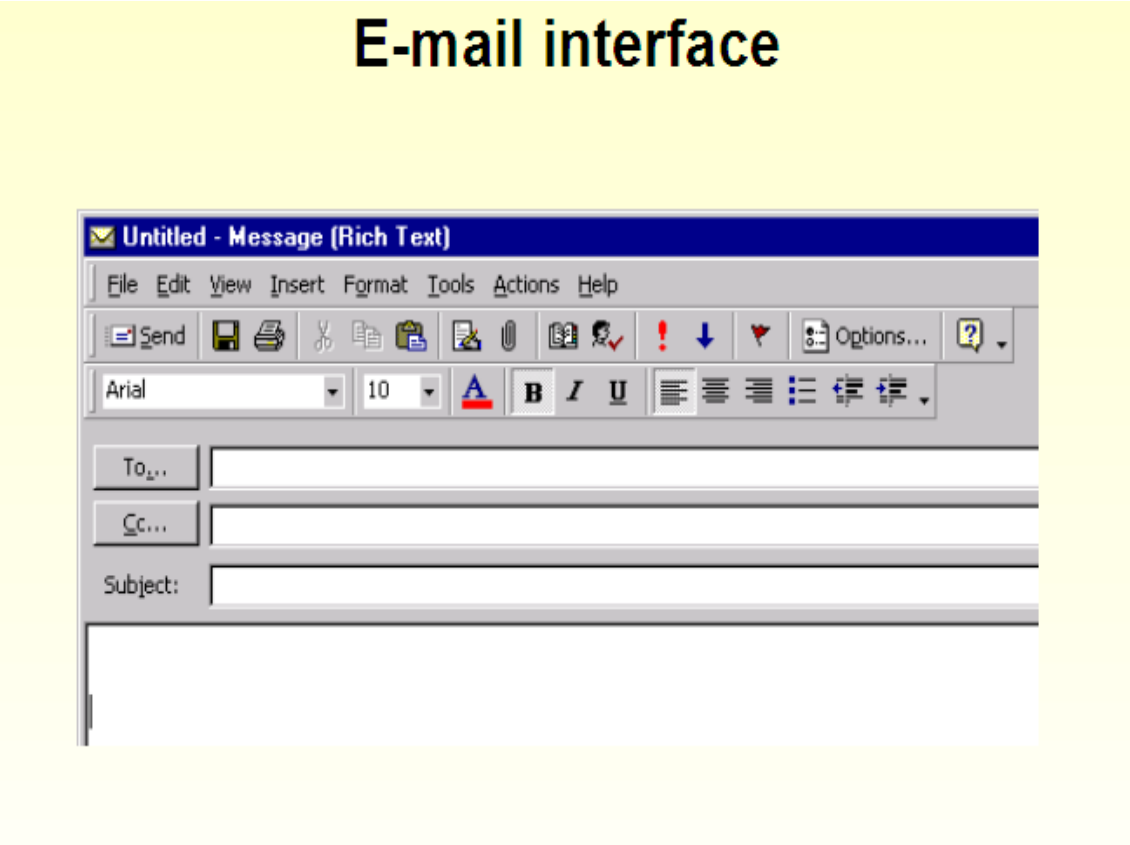
Picture 11: Object destruction

Now we will consider a scenario for sending email and learn how to draw the sequence diagram for that.



Picture 12: An Email client

We are considering a email application similar to the one shown below.



Picture 13: Email Interface

The steps involved in sending an email is given in the picture below.

Working on a scenario-Sending an email

1. Click on the 'New' email icon

2. Enter the sender's email id in the To section

3 Enter the subject line

4. Enter the message in the body of the email

5.Click on the send button.

6.The system checks for email address from the address book.

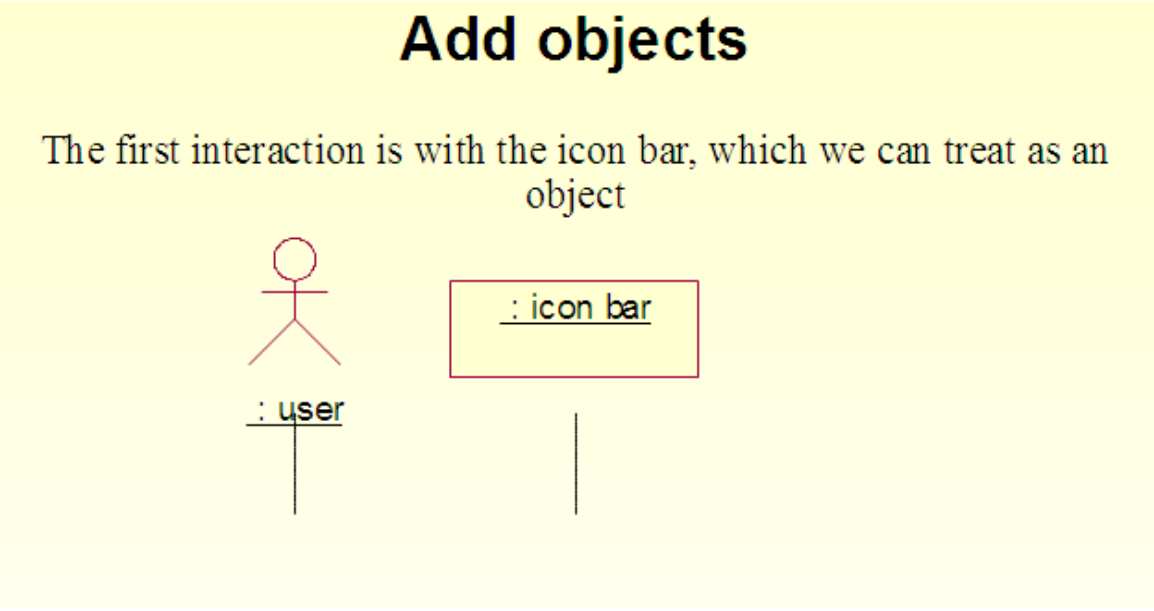
7.Finally, the system submits the email to the email server.

Picture 14: Steps involved in sending email

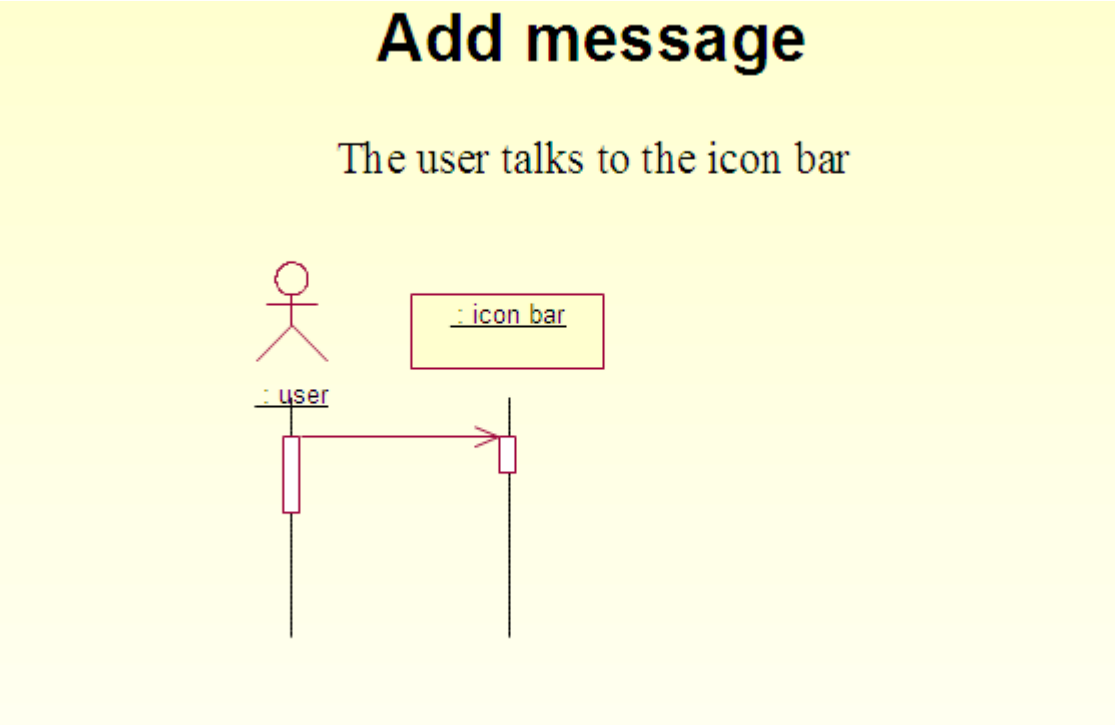
Now we will see how to go about drawing the sequence diagram.

Picture 15: First step in sequence diagram

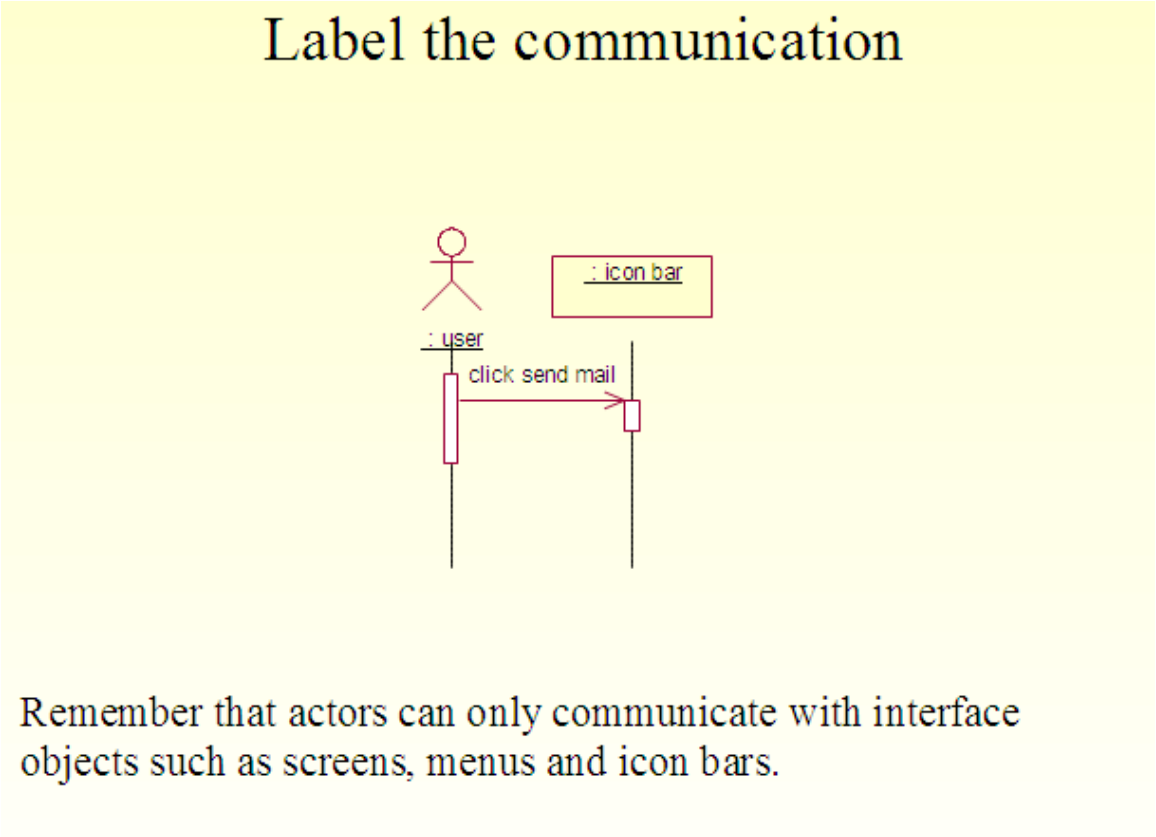
The first step will be to click the new email icon. How to represent this using UML notation is shown in the diagram given below.



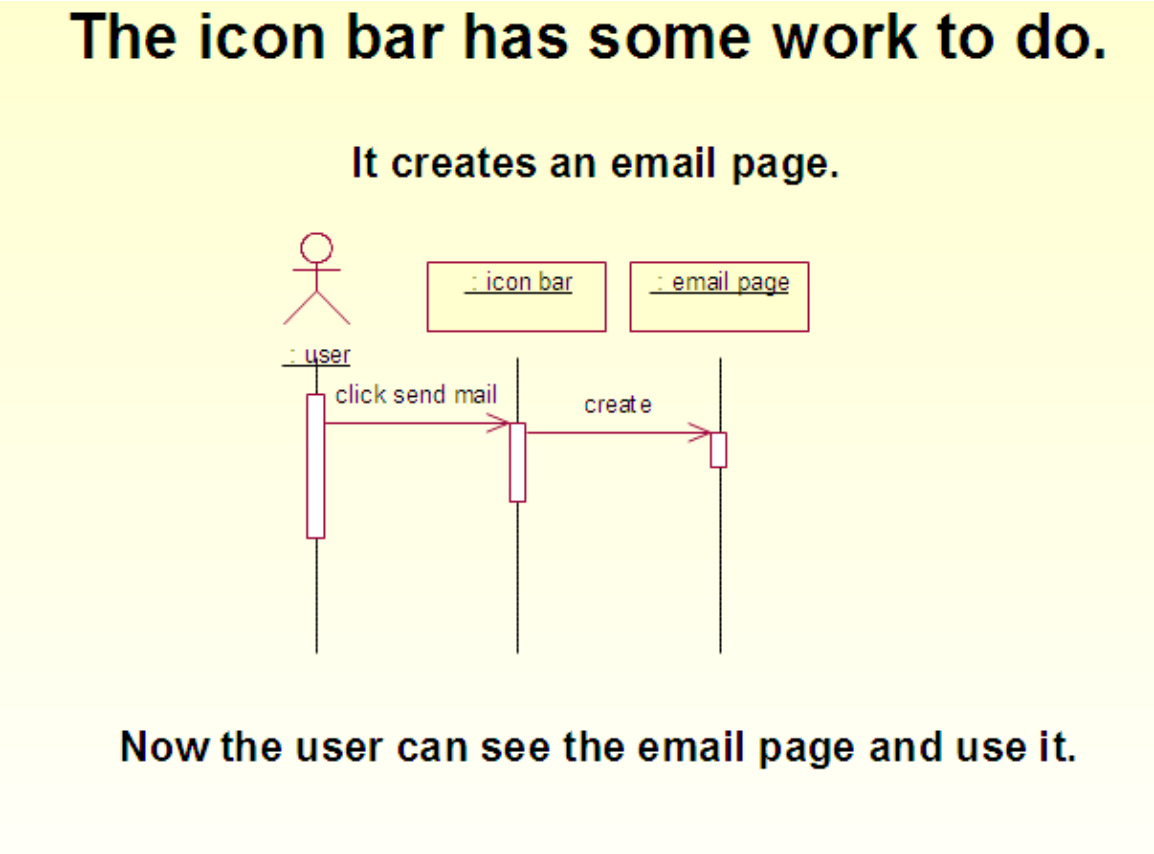
Picture 16: Actor clicks the new mail icon



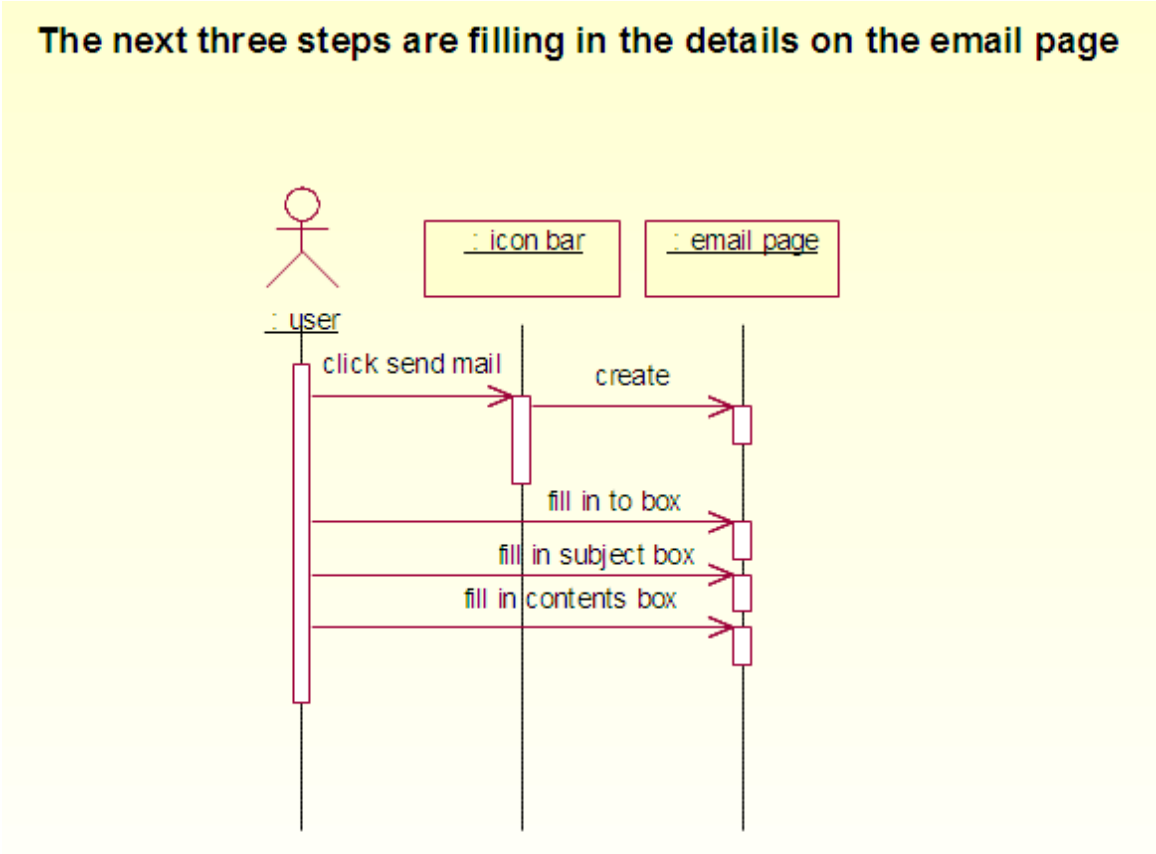
Picture 17: UML notation for communication between actor and icon



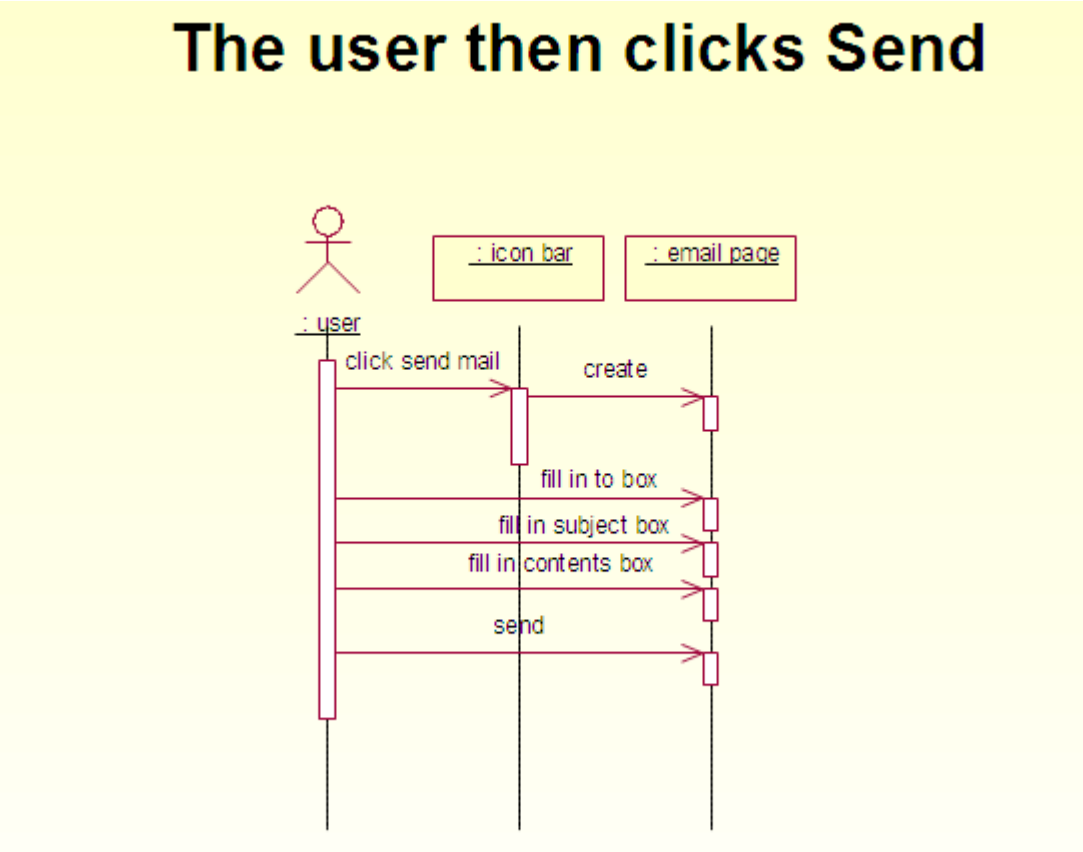
Picture 18: UML notation to represent the information sent between actor and the icon



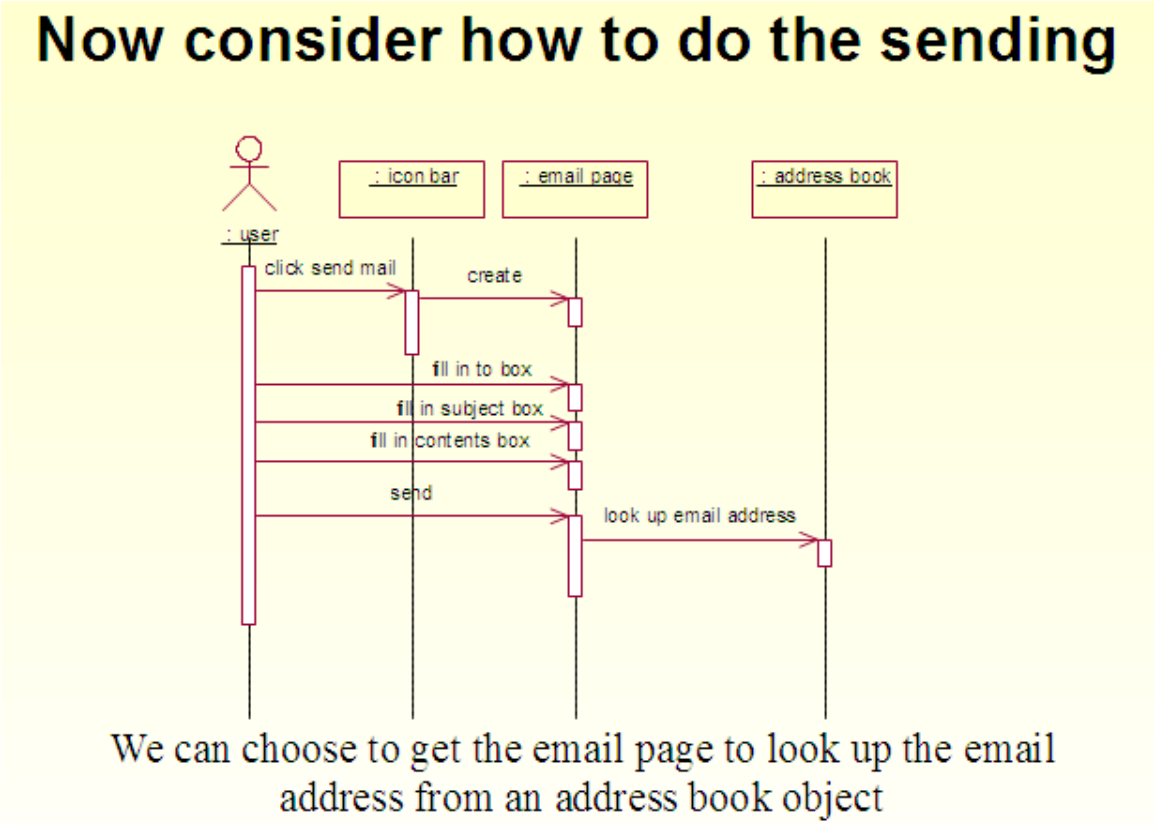
Picture 19: UML notation to indicate that a new email is created.



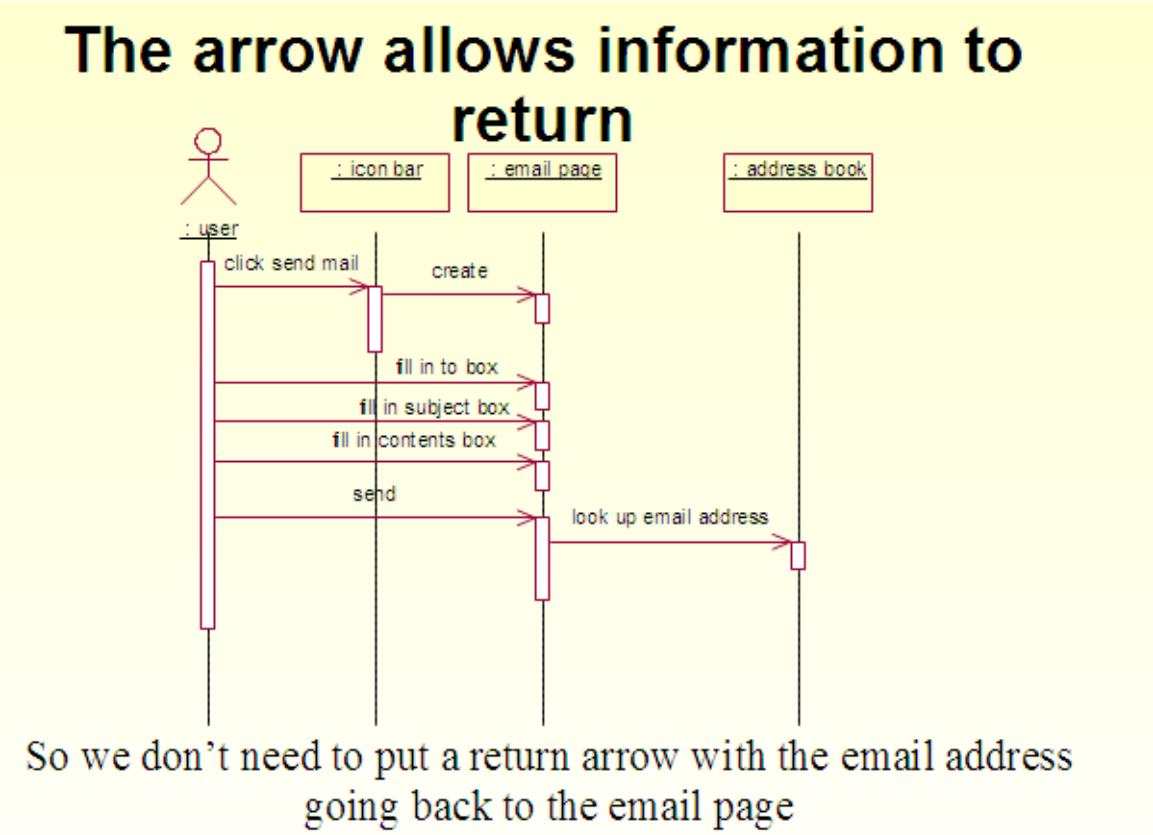
Picture 20: UML notations to indicate the information that need to be entered for sending email



Picture 21: UML notation to indicate that the user is clicking send button

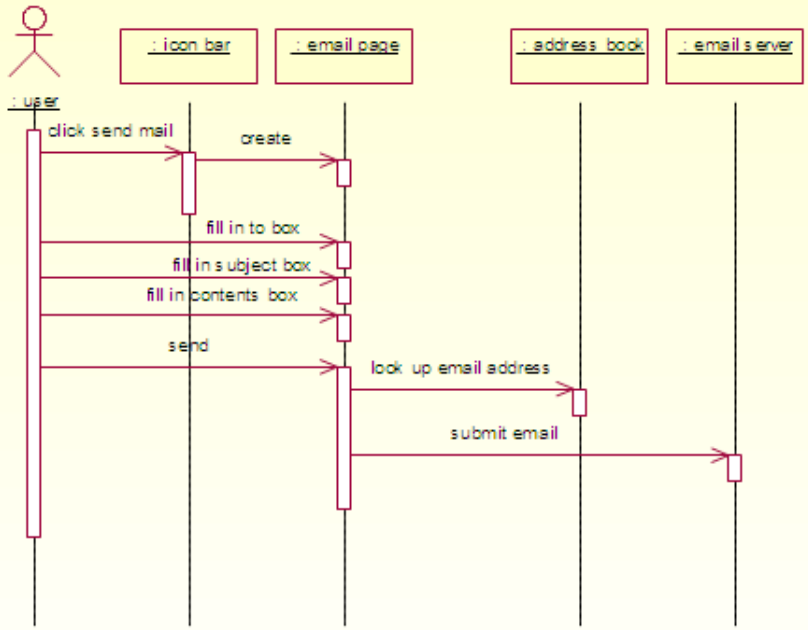


Picture 22: UML notation to represent searching in the address book for recipient address



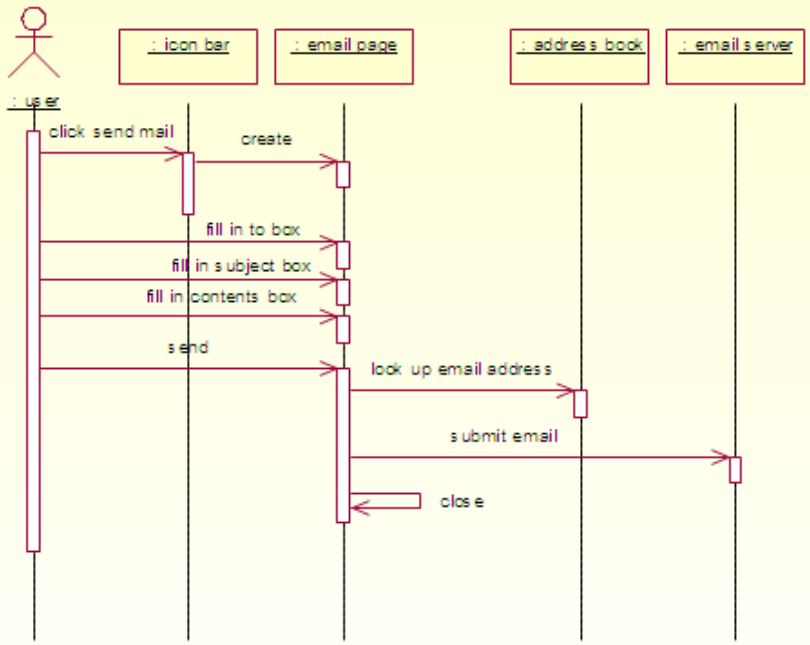
Picture 23: UML notation to represent looking up of email address

We can choose to get the email page to submit the email to the email server

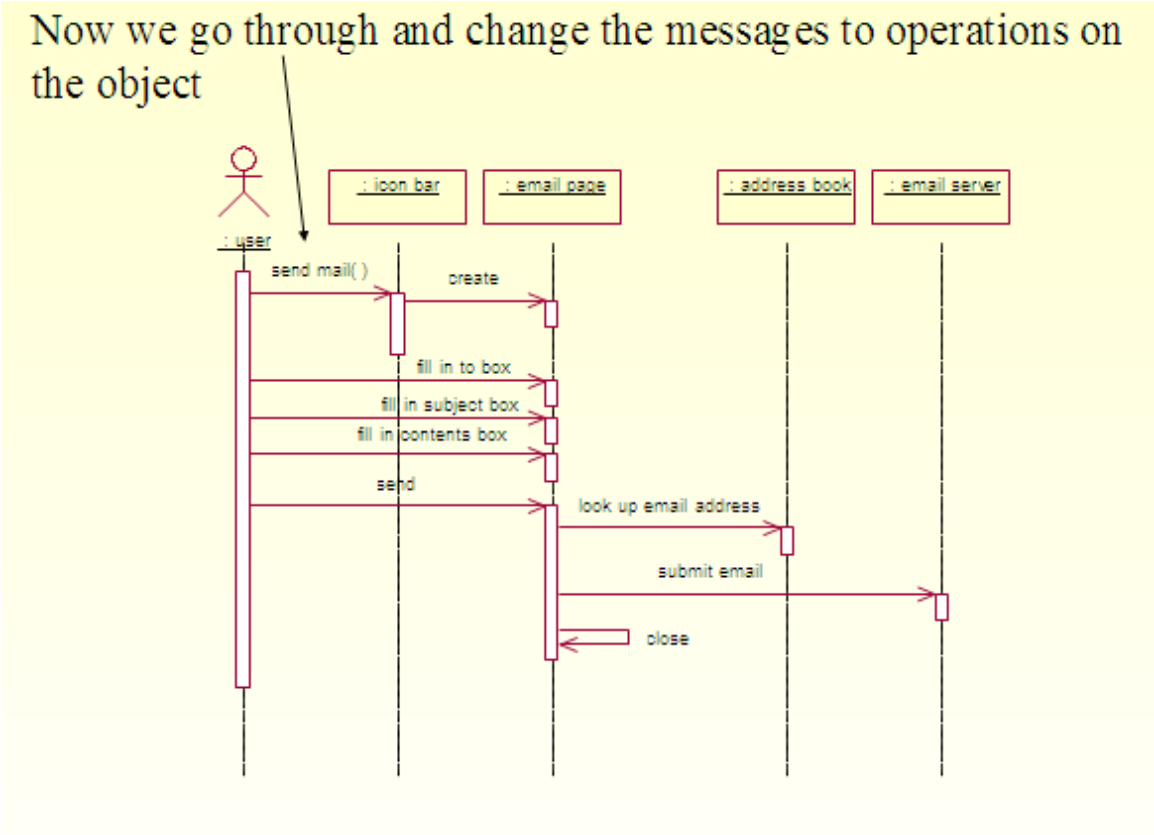


Picture 24: UML notation to indicate that the mail is sent to the server

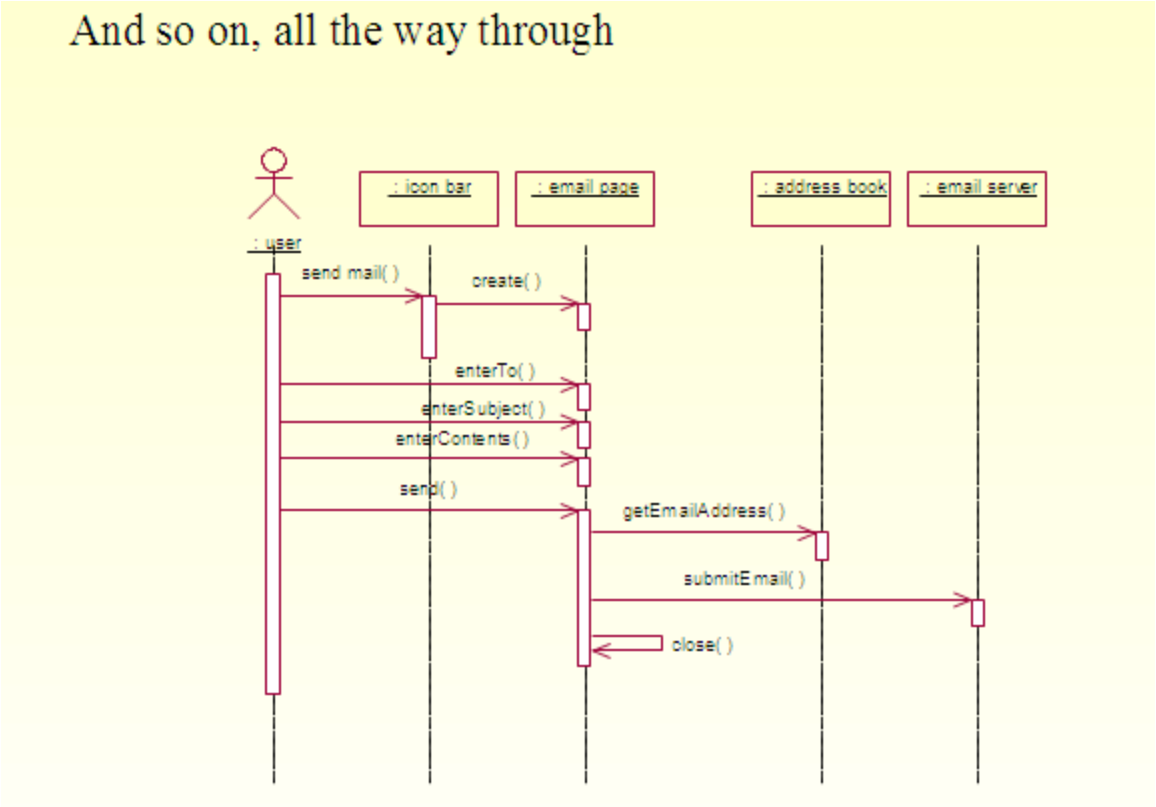
And if we think carefully, the email page always closes after the send.



Picture 25: UML notation to indicate the closing of email page



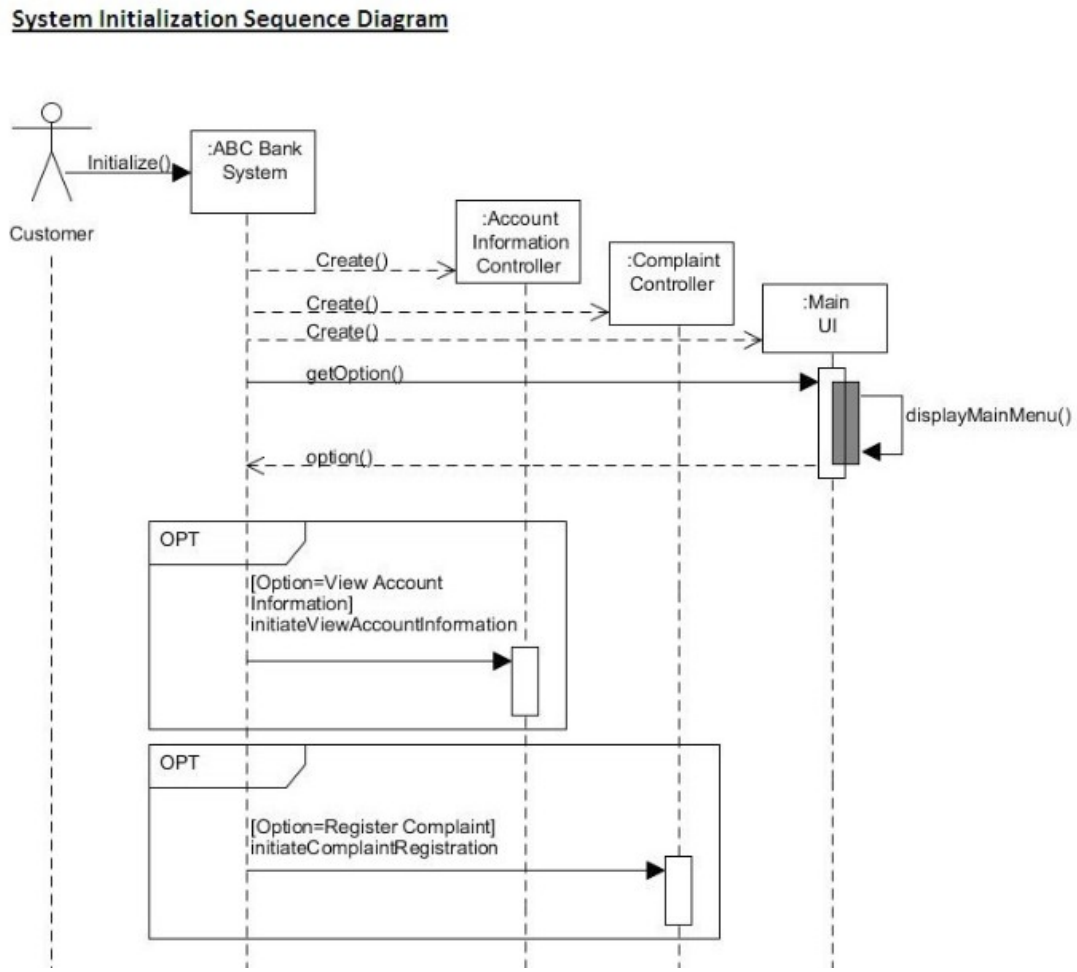
Picture 26: UML notations to convert textual representations to method calls



Picture 27: Final UML diagram for email sending scenario

Now, we will try to solve a complaint registration scenario for ABC bank using MVC architecture. First, comes the system initialization diagram and then followed by the sequence diagram for complaint registration.

System initialization diagram is used to represent the initial state of the ABC bank system. In this diagram, all the controllers involved in the system, along with the main UI page is shown. The option available in the main UI is also shown. The Actor can select any option. It is necessary to have a sequence diagram corresponding to each of the option listed in the system initialization diagram. Here we are focusing only on the complaint registration sequence diagram.

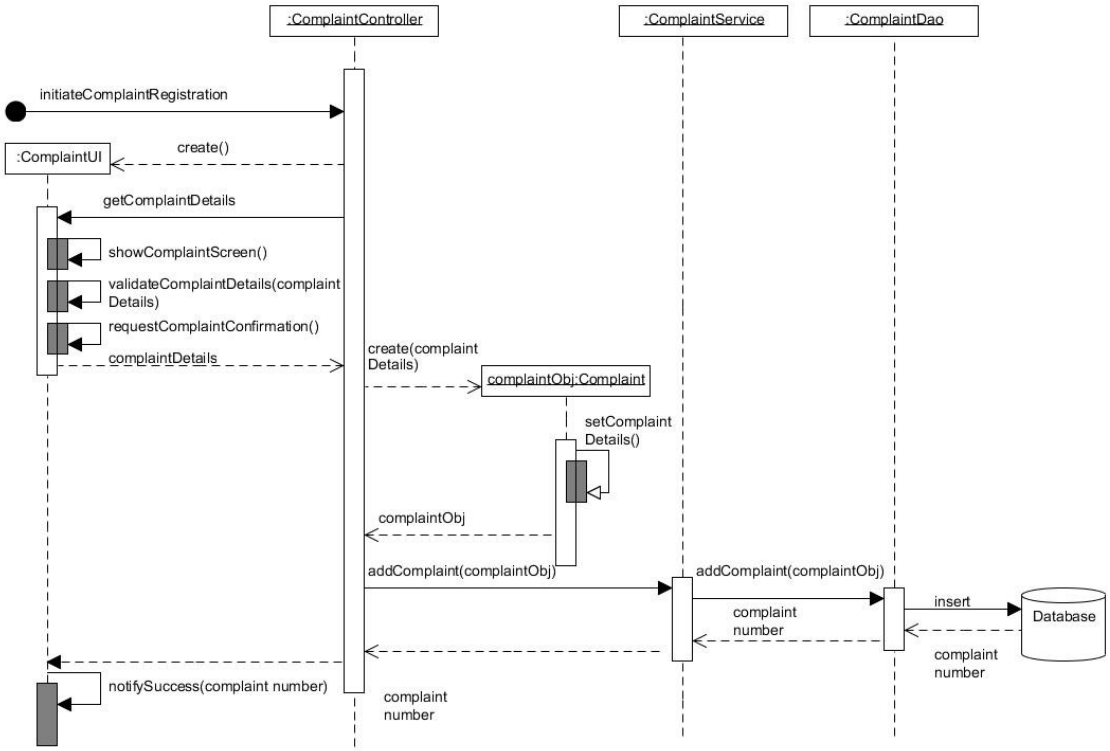


Picture 28: System initialization diagram for ABC bank

In MVC architecture, following are the steps:

- The actor initiates a request by selecting an option available in the system.
- The actor initiated request is transferred to the controller.
- The controller brings up the corresponding UI.
- Actor enters the details in the UI. Necessary client side validations(using javascript) is done.
- If client side validation is completed successfully, the details are passed on to the controller.
- Controller sets the details of the complaint to the bean object(by using constructor or by using setters).
- The bean object is passed on to the Model(Business logic or the Service layer) using the addComplaint method.

- The bean object is then passed to the ComplaintDAO for inserting the record into the database.
- The complaintId generated is passed back to service layer, then to controller and eventually to the UI.



Picture 29: Complete sequence diagram for complaint registration process.

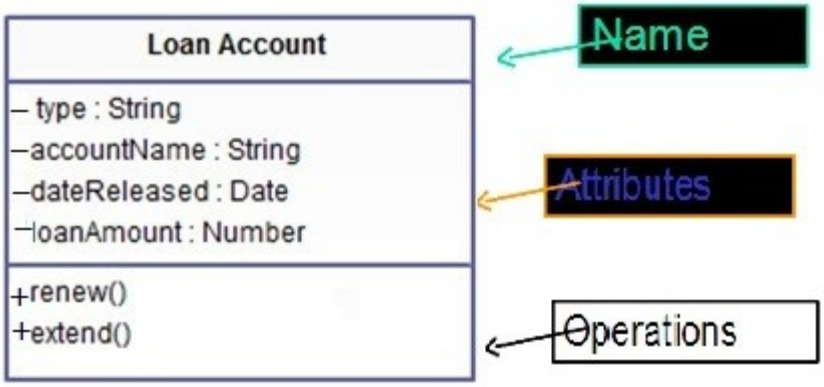
Now, revisiting our email scenario, for explaining the class diagram.

1.2.3.4.4 Design Level Class Diagram

By definition, a class diagram is a diagram which shows a collection of classes and interfaces, along with the collaborations and relationships among classes and interfaces.
A class diagram is a pictorial representation of the detailed system design. Design experts who understand the rules of modeling and designing systems designs the class diagrams. An important thing to be noted is that a class diagram is a static view of a system. The structure of a system is represented using class diagrams. Class diagrams are references for the the developers during the implementation of this system.
Now you now know what a class diagram is. But, how does a class diagram relate to the use case diagrams that you read in the previous article? When you designed the use cases, you must have realized that the use cases deals with the requirements of a system. The aim of designing classes is to convert this "what" to a "how" for each requirement. Each use case is further analyzed and broken up into atomic components that form the basis for designing the classes.

Each class is represented by a rectangle. They are subdivided into three compartments

- Name
- Attributes
- Operations

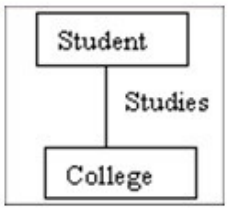
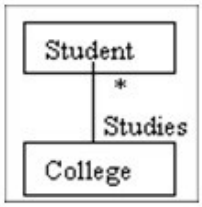
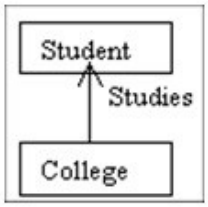


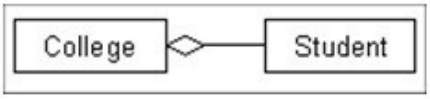
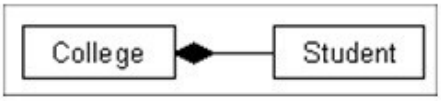
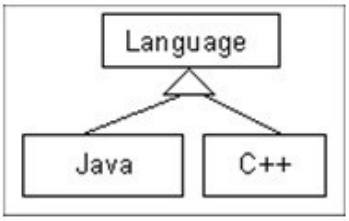
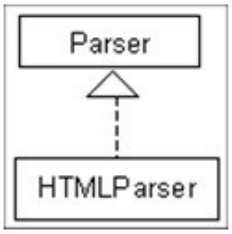
Simple class diagram with attributes and operations

Modifiers are used to indicate visibility of attributes and operations.
'+' is used to denote *Public* visibility (everyone)
'#' is used to denote *Protected* visibility (friends and derived)
'-' is used to denote *Private* visibility (no one)

1.2.3.4.5 4. Relationships Between Classes

In a class diagram, classes should have a relationship with each other.. The following table shows the kinds of relationships between classes, their notations, and the detailed description of the relationship.

Sr. No.	Relation	Symbol	Description
1	Association		When two classes are connected to each other in any way, an association relation is established. For example: A "student studies in a college" association can be shown as:
1 a.	Multiplicity		An example of this kind of association is many students belonging to the same college. Hence, the relation shows a star sign near the student class (one to many, many to many, and so forth kind of relations).
1 b.	Directed Association		Association between classes is bi-directional by default. You can define the flow of the association by using a directed association. The arrowhead identifies the container-contained relationship.

2	Aggregation		When a class is formed as a collection of other classes, it is called an aggregation relationship between these classes. It is also called a "**has a**" relationship.
2 a.	Composition		Composition is a variation of the aggregation relationship. Composition connotes that a strong life cycle is associated between the classes.
3	Inheritance/Generalization		Also called an "**is a**" relationship, because the child class **is a** type of the parent class. Generalization is the basic type of relationship used to define reusable elements in the class diagram. Literally, the child classes "inherit" the common functionality defined in the parent class.
4	Realization		In a realization relationship, one entity (normally an interface) defines a set of functionalities as a contract and the other entity (normally a class) "realizes" the contract by implementing the functionality defined in the contract.

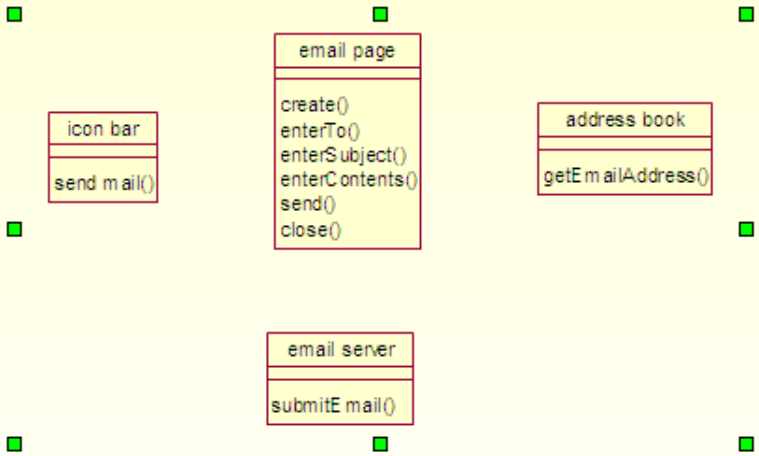
Multiplicity Indicators

Exactly one	1
Zero or more (unlimited)	* (0..*)
One or more	1..*
Zero or one (optional association)	0..1
Specified range	2..4
Multiple, disjoint ranges	2, 4..6, 8

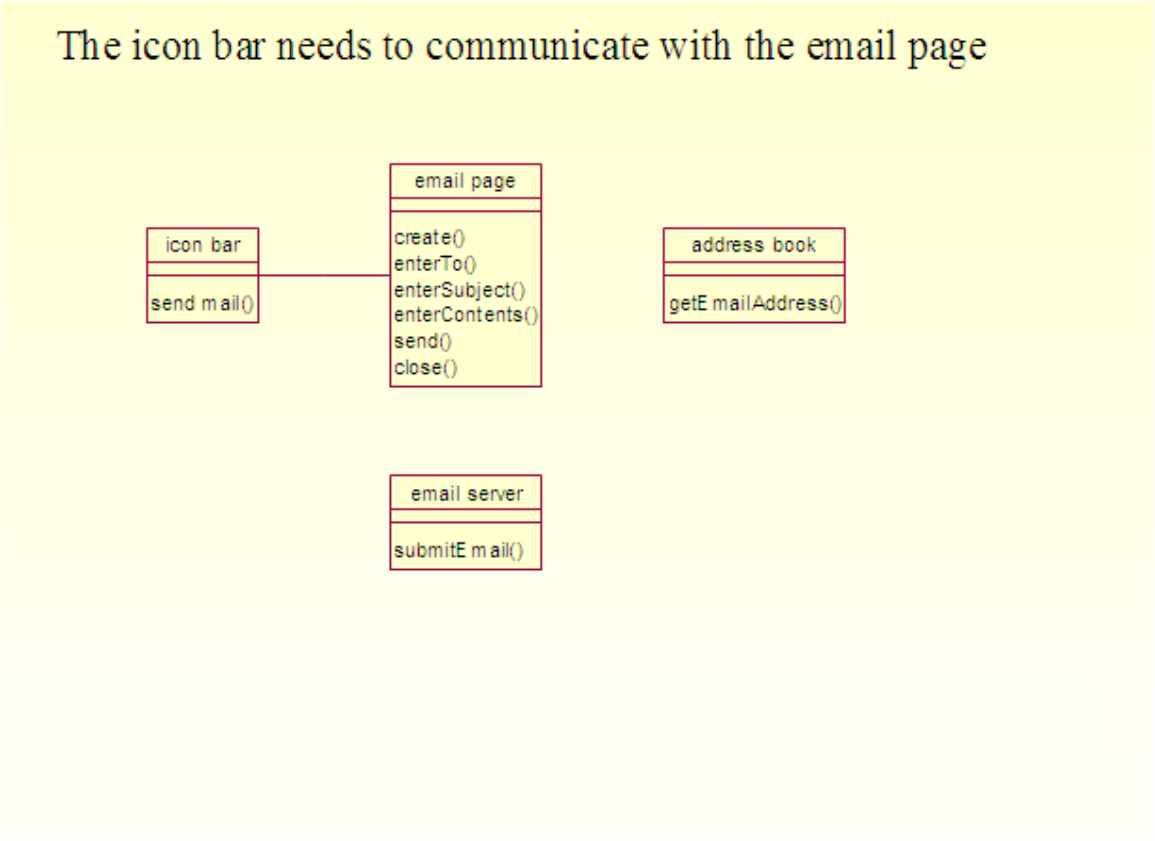
Picture 30: UML notations for multiplicity

Let us now try to draw the class diagram for a email scenario that we discussed while drawing the sequence diagram.

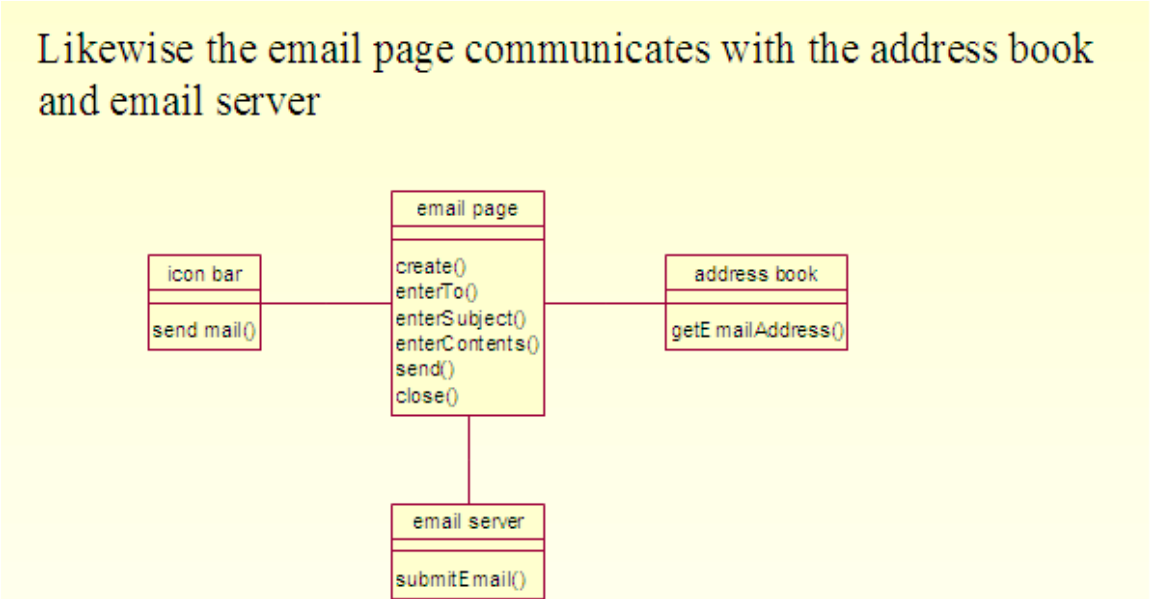
In sequence diagram section for the example of sending email we have found our objects, and the operations on them.Now we Will make the class diagram for that.



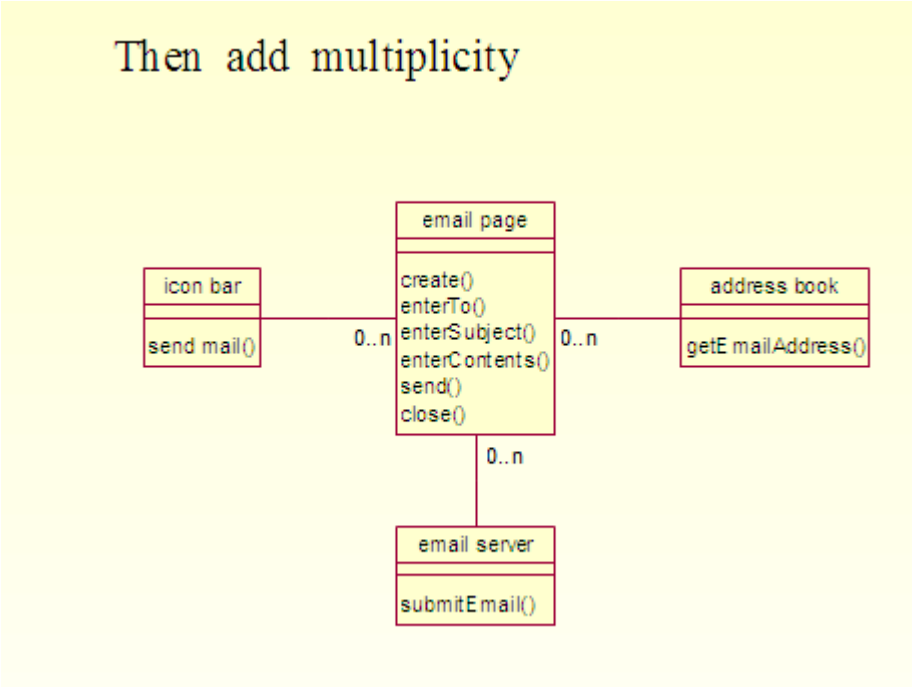
Picture 31: Classes identified for Email scenario



Picture 32: UML notation to indicate relation between classes

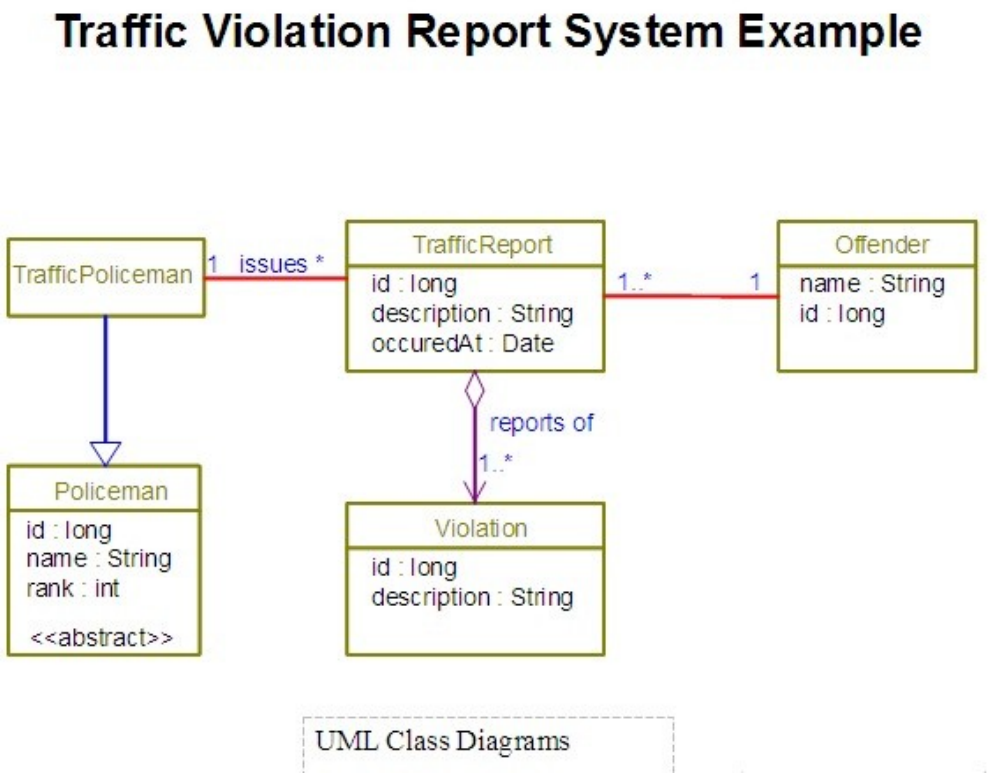


Picture 33: Establishing relation between all objects



Picture 34: Adding multiplicity to the relations identified

In the below sample for Class Diagram, try to identify the relationship between the classes.



Picture 35: Assignment

1.2.3.5 Deliverables at the end of Design Phase

Deliverables at the end of Design phase are:

- High Level Design Document
- Low level Design Document(contains sequence diagram and Design level class diagram)

1.2.3.6 Assignment <Optional>

For the use cases identified as part of assignment given , try to draw the sequence diagram and the design level class diagram(by following MVC architecture). It is suggested to practice drawing atleast one sequence diagram using violet or eclipse for practice. This will be required for PD phase.

1.2.3.7 Summary

To summarize, following are the activities that happen in a project as part of SDLC. The deliverables at the end of each phase is also given in the diagram below.

Appendix -1 Reference Template

SRS Template